Computer Networks 289 (2026) 112739

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

Ultra fast encrypted network traffic classification based on classification by retrieval powered by out-of-distribution

Yehonatan Zion [a,*], Amit Dvir [a], Chen Hajaj [b], Ran Dubin [a], Neriya Filber [c]

[a] Department of Computer and Software Engineering, Ariel Cyber Innovation Center, Ariel University, Israel
[b] Department of Industrial Engineering and Management, Data Science and Artificial Intelligence Research Center, Ariel University, Israel
[c] Department of Computer Science, Ariel Cyber Innovation Center, Ariel University, Israel

ARTICLE INFO

Keywords:
Encrypted network traffic classification
Classification by retrieval
Approximate nearest neighbor
Out-of-distribution detection
Few-shot learning
Network security

ABSTRACT

The dominance of encryption and the relentless growth of network traffic have rendered traditional deep packet inspection ineffective and made retraining-based machine learning classifiers operationally costly, as every new application or threat class requires a full model rebuild. In order to overcome this challenge, we present an ultra-fast encrypted traffic classification system based on Classification By Retrieval (CBR): each flow is represented by a fixed-length feature vector extracted from only the first few packets, indexed in a HNSW approximate nearest-neighbor structure, and classified by retrieving its nearest labeled neighbor. New classes are absorbed by inserting a handful of labeled vectors into the running index, with no retraining and no downtime, and out-of-distribution traffic is detected within the same retrieval step using a single global distance threshold. We evaluate handcrafted statistical features and learned CNN and MLP embeddings against a Random Forest baseline on the public CESNET-TLS22 dataset (184 application classes) and a proprietary cellular dataset (RADCOM). With CNN embeddings, CBR matches Random Forest accuracy (macro F1 of 0.923 versus 0.925 on CESNET) while training roughly ten times faster, adapts to new classes two to three orders of magnitude faster (seconds versus thousands of seconds), and detects out-of-distribution flows at up to 0.92 AUROC. The retrieval index is also far cheaper to keep resident than the baseline it replaces, holding 1.2 million CESNET flows in 171 MB against 1,350 MB for the serialized forest, because its size grows with the number of indexed flows rather than with the number of classes. The Random Forest remains the faster classifier per flow, so the trade-off is per-flow latency against adaptability and memory. These results show that retrieval-based classification is a practical and adaptable alternative for real-time encrypted traffic analysis in dynamic network environments.

1. Introduction

The analysis of internet traffic has grown increasingly complex due to two primary factors: the widespread adoption of encryption protocols and the exponential increase in traffic volume. Encryption now dominates the majority of internet traffic, largely driven by the adoption of standards such as Transport Layer Security (TLS). This trend reflects a broader industry movement toward secure connections spurred by growing privacy and security awareness. Initiatives like Let us Encrypt [1] have further supported this shift, enabling more websites and services to adopt encryption seamlessly. While encryption enhances privacy and security for legitimate users, it also introduces significant challenges for traditional Deep Packet Inspection (DPI) traffic analysis methods. As a result, understanding and monitoring encrypted traffic have become critical focus areas for network security and traffic analysis efforts.

In recent years, significant changes to internet protocols, such as QUIC [2], HTTP/3, TLS 1.3, and DNS over HTTPS (DoH) [3], have further complicated traffic classification. Traditional DPI methods, which rely on identifiers like DNS queries and Server Name Indication (SNI) to analyze encrypted traffic, are quickly becoming obsolete. Machine Learning (ML) models have emerged as a potential solution for encrypted traffic classification. However, these models require extensive labeled datasets for training and can only classify data into pre-defined categories. When new applications or traffic types emerge, these models become outdated. Updating them involves collecting data for the new category, integrating it into the existing dataset, and retraining the

* Corresponding author.
E-mail addresses: yehonata.zion@msmail.ariel.ac.il (Y. Zion), amitdv@ariel.ac.il (A. Dvir), chenha@ariel.ac.il (C. Hajaj), rand@ariel.ac.il (R. Dubin), neriya.filber@msmail.ariel.ac.il (N. Filber).

https://doi.org/10.1016/j.comnet.2026.112739
Received 16 June 2026; Received in revised form 22 August 2026; Accepted 31 August 2026
Available online 9 September 2026
1389-1286/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

model, a resource-intensive process that poses significant operational challenges.

The dramatic increase in Internet traffic volume presents another significant challenge. A recent Ericsson study [4] reported that global cellular network data traffic reached 145 exabytes (EB) per month at the end of Q1 2024, marking a growth 6% quarter-over-quarter from Q4 2023. Video content alone accounted for 73% of all cellular data traffic at the close of 2023. This relentless growth, projected to continue at an average annual rate of 20% until 2029, underscores the need for faster, more accurate traffic analysis methods. Classifying this amount of data in real time is a new challenge.

Taken together, these trends define the three scientific problems that this work addresses, summarized in Fig. 1. The first is *adaptation*: the traffic a network carries does not hold still. New applications and threats appear, and the applications already known drift, since an update or a protocol change alters how a class looks on the wire even though its label has not changed. A trained classifier is closed over a fixed set of classes learned at one moment in time, so both forms of change force the same full rebuild. The second is *the unknown*: a flow may belong to no known class at all, and a closed-set classifier is obliged to assign it one of the labels it already has. The third is *cost*: whatever the remedy for the first two, it must not become more expensive to apply than the traffic is large. At 145 EB of cellular traffic per month, a remedy that retrains over the accumulated corpus every time the class set moves, and that stores a model growing with the class count, is one a deployment cannot afford to apply often enough to keep up. The first two are precisely the problems that recent surveys of encrypted traffic classification single out as inadequately addressed, namely adaptation without frequent retraining and the detection of unknown traffic [5,6]. A fourth, cross-cutting question runs through all three: which flow representation a retrieval index should be built on in the first place, which we settle with a like-for-like comparison rather than by convention.

To address these challenges, our approach performs Classification By Retrieval (CBR): it leverages approximate nearest-neighbor (ANN) search to efficiently classify new vectors by matching them to a prelabeled, indexed collection. This enables real-time classification and allows the system to incorporate new traffic classes without requiring retraining. If the distance of a sample to all known classes exceeds a predefined threshold, it is classified as out-of-distribution (OOD), thereby facilitating the detection of emerging or unknown traffic patterns. The system can dynamically expand with minimal labeled data from new classes, ensuring adaptability. Furthermore, by relying solely on statistical features, it remains robust against potential protocol changes.

1.1. Contributions

While recent literature highlights that closing the gap between static models and evolving operational environments remains an open challenge, our novelty lies in a unified framework designed specifically for seamless deployability. In summary, the contributions of this paper are as follows, each labeled with the badge under which it appears in Fig. 1:

- **(C1) A retraining-free retrieval framework for near-real-time concept-drift remediation**, addressing P1. We represent each flow by a fixed-length vector extracted from only the first few packets, and classify it by ANN retrieval over a labeled Hierarchical Navigable Small World (HNSW) [7] index. New samples and entirely new classes are absorbed by inserting their vectors into the running index at $\mathcal{O}(\log n)$ cost (with n the number of indexed flows), enabling continuous, live index updates without gradient updates, model retraining, or system downtime.
- **(C2) Unified, training-free out-of-distribution detection**, addressing P2. OOD flows are flagged within the same retrieval step using a single global distance threshold that is distribution-free and requires no per-class statistics. This unifies classification and OOD detection into a single operational step, eliminating the complex per-class bookkeeping or extra-class training common in existing methods.
- **(C3) Substantial efficiency gains in training and adaptation**, addressing P3. On a State-of-the-art (SOTA) public dataset (CESNET-TLS22), CBR matches the Random Forest (RF) baseline in accuracy (CNN-CBR macro F_1 of 0.923 versus 0.925) while training roughly ten times faster, and adapts to new classes two to three orders of magnitude faster (seconds versus thousands of seconds).
- **(C4) A comprehensive comparison across the two ends of the feature spectrum**, addressing the cross-cutting representation question. We evaluate handcrafted (HC) statistical features, and learned Convolutional Neural Network (CNN) and Multi-Layer Perceptron (MLP) embeddings, each against an RF baseline, across packet budgets, few-shot regimes, and OOD detection, on two SOTA public datasets (CESNET-TLS22 with 184 classes, and MIRAGE-2019, which additionally enables comparison against published few-shot baselines) and a real-world industry dataset (the proprietary cellular RADCOM dataset), with a fourth corpus of malware traffic serving as a cross-dataset OOD probe.

List of Abbreviations

ANN	Approximate Nearest Neighbor
AUROC	Area Under the ROC Curve
CBR	Classification By Retrieval
CNN	Convolutional Neural Network
DoH	DNS over HTTPS
DPI	Deep Packet Inspection
HC	Handcrafted
HNSW	Hierarchical Navigable Small World
kNN	k-Nearest Neighbor
ML	Machine Learning
MLP	Multi-Layer Perceptron
NN	Nearest Neighbor
OOD	Out-of-Distribution
RF	Random Forest
ROC	Receiver Operating Characteristic
SNI	Server Name Indication
SOTA	State-of-the-Art
TLS	Transport Layer Security

2. Related work

This section provides a overview of existing work in encrypted network traffic classification and positions our framework within the current research landscape. To systematically contextualize our contribution, this section categorizes the existing literature into three main research directions: (1) feature representation paradigms, (2) few-shot and class-incremental learning, and (3) out-of-distribution (OOD) detection. Furthermore, we explicitly survey the under-explored domain of retrieval-based classification, positioning our framework's specific breakthroughs against the common defects of existing methods. Before delving into each category, we first outline the fundamental limitations of traditional approaches that motivate our work. Machine Learning and Deep Learning have become the dominant approaches for encrypted network traffic classification [8–14]. Despite strong classification performance, these works share a fundamental limitation: they

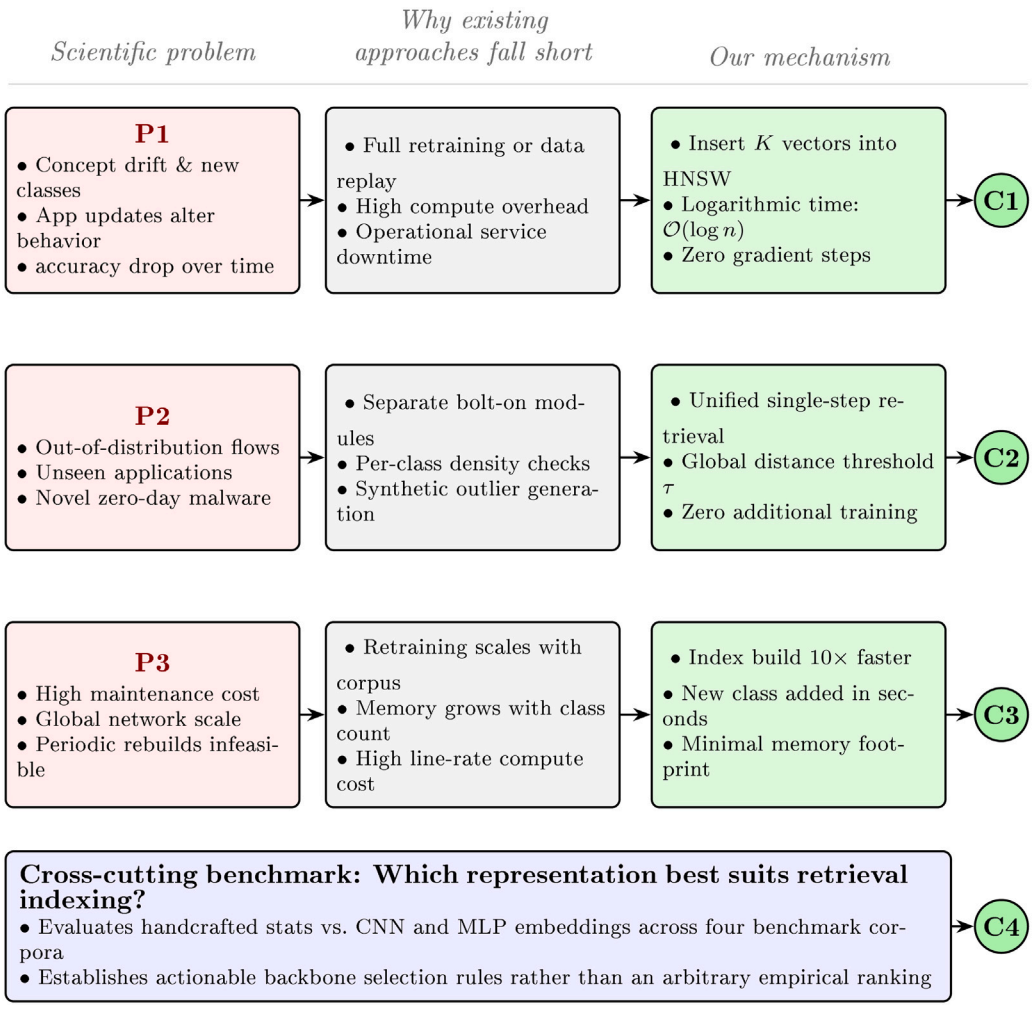


Fig. 1. The scientific problems this paper addresses. Each row reads left to right: the problem (P1–P3), the reason existing encrypted-traffic classifiers fall short of it, and the mechanism by which our retrieval framework resolves it. The badge on the right names the contribution of Section 1.1 that the row delivers. P1 and P2 are precisely the two open problems that recent surveys of the field identify as inadequately addressed, namely adaptation without frequent retraining and the detection of unknown traffic [5,6], and the two-year accuracy loss quoted in P1 is reported in [5]. The traffic volume in P3 is the Q1 2024 cellular figure of [4], and the efficiency, memory and adaptation figures behind the right-hand column are established in Section 5. The bottom band is the cross-cutting representation study rather than a fourth problem. Fig. 2 then details how the mechanisms in the right-hand column are realized.

treat the class set as fixed, and require full model retraining whenever new traffic classes appear. This is a costly and operationally impractical assumption in dynamic network environments. The severity and real-world implications of this limitation are strongly highlighted in recent literature. Recent comprehensive surveys quantify the impact of traffic drift on long-term model resilience. For instance, classifier accuracy can drop by more than 40% over a two-year window due to traffic drift [5]. Furthermore, both surveys identify two critical open problems left inadequately addressed by current approaches: adaptation without frequent retraining, and the detection of unknown traffic [5,6]. Notably, neither survey considers approximate-nearest-neighbor index retrieval as part of the solution space which represents the exact gap our framework addresses.

2.1. Feature representation paradigms

Retrieval-based classification bypasses the retraining requirement by classifying new queries against a pre-indexed reference set, making feature representation a central design decision. We therefore survey the two dominant feature paradigms explored for this task: HC statistical vectors and end-to-end learned embeddings.

Biernacki [15] demonstrates that statistical throughput features such as average bit rate, variance, and extrema are highly discriminative for encrypted video traffic classification, confirming the value of flow-level statistics over raw-payload inspection. Zhao et al. [16] expose fundamental limitations of deep representation learning for encrypted traffic: models trained on per-packet data splits exploit shortcut features such as TCP sequence numbers, and their accuracy collapses below 40% when evaluated correctly on per-flow splits. Shallow models built on expert-selected statistical features consistently match or outperform large Transformer- and Mamba-based encoders at a fraction of the computational cost. These findings confirm that HC statistical vectors are a robust baseline. At the same time, end-to-end learned representations can capture non-linear inter-feature interactions that expert statistics may miss. We therefore evaluate both feature regimes in our experiments.

2.2. Retrieval-based traffic classification

While retrieval-based classification has been highly successful in domains like computer vision and NLP, its specialized application to encrypted traffic classification remains heavily under-explored. Consequently, there is a lack of extensive literature dedicated exclusively to this paradigm within our domain.

Ma et al. [17] provide one of the few notable exceptions, applying a weighted k-NN approach with self-adaptive feature importance to fine-grained encrypted traffic classification using statistical flow features. However, a primary drawback of this method is that it addresses neither new-class adaptation, OOD detection, nor indexing scalability.

Because retrieval methods require a strong supervised baseline for meaningful comparison, we adopt Random Forest. A substantial body of encrypted-traffic literature establishes it as a superior alternative to deep models rather than a weak reference point: RF outperforms CNNs and LSTMs on QUIC traffic [18], matches or exceeds 94% accuracy across various benchmarks [19,20], and remains superior even under the stricter Encrypted ClientHello (ECH) setting, where DL classifiers analyzing raw byte sequences mistake encryption-induced noise for signal, with a hybrid RF approach over TLS metadata and flow statistics reaching a 94.6% macro F-score [21]. Random Forest is therefore the prevailing high-performance classical method, making it a demanding reference point against which to position our retrieval-based approach.

2.3. Few-shot and class-incremental learning

To address the new-class adaptation gap, recent work has explored few-shot, metric, and class-incremental learning. Bovenzi et al. [22] evaluate multiple paradigms on rare mobile-app traffic [23]. Their transfer-learning variants require gradient updates to adapt. Their metric-based variants (ProtoNet [24], MatchingNet [25]) avoid gradient steps but still rely on specialized episodic meta-training over base-class tasks. Li et al. [26] propose CL-MetaFlow, a two-stage training procedure before the encoder is frozen, which still handles new classes by recomputing per-class prototype vectors. Zhao et al. [27] freeze a large backbone but require retraining a lightweight contrastive module whenever new classes emerge. Crucially, this refresh requires retaining base-class samples. Yang et al. [28] rely on episodic multi-task meta-training followed by gradient fine-tuning. Di Monda et al. [29] avoid gradient steps by explicitly matching against a support set, yet still rely on episodic meta-training and exclude OOD detection entirely.

A parallel line approaches the new-class problem through class-incremental learning. Cai et al. [30] update a contrastive prototype network on a mixture of new classes and reserved exemplars of old classes, reporting that accuracy on old classes falls from 95.4% to 91.4%. ILETC [31] retrains a classifier with a generative model that replays synthetic samples, and its accuracy drops by roughly ten points once all classes are added.

A common limitation systematically shared across all these few-shot and class-incremental methods is their ultimate reliance on either episodic meta-training, gradient-based fine-tuning, prototype recomputation, or the generative replay of prior-class data. Furthermore, none of these methods unifies OOD detection directly into the classification step.

2.4. Out-of-Distribution (OOD) detection

A related challenge arises when a query comes from a class entirely outside the known distribution. Ansari et al. [32] propose synthesizing in-distribution samples with added noise to train an "Oracle" model. Zhao et al. [27] compute a per-class OOD threshold from the class centroid and variance in a contrastive embedding space.

A common limitation of these approaches is their architectural fragmentation: they treat OOD detection as an isolated task requiring explicit auxiliary training (e.g., synthetic sample generation) or complex per-class statistical bookkeeping. Rather than aiming to introduce a novel OOD detection algorithm, our objective is to demonstrate that a fundamental OOD capability can be organically unified into the classification process without these overheads.

2.5. Positioning of our framework

Our system differs fundamentally from all of the above, addressing their structural defects (see Section 1.1): the feature backbone is trained *once* on the base classes and then frozen permanently. New classes, even from a single labeled sample, are absorbed by inserting their individual embeddings directly into the ANN index in $\mathcal{O}(\log n)$ time. This breakthrough completely eliminates gradient steps, requires no access to prior training data, and avoids per-class centroid recomputation. Furthermore, rather than relying on external OOD modules, our framework unifies classification and OOD detection within the exact same HNSW retrieval step by applying a single global ANN distance threshold, effectively achieving OOD awareness with zero additional training overhead [33].

To the best of our knowledge, no prior encrypted-traffic system combines all of these properties in a single framework. Table 1 systematically summarizes this comparison, highlighting how each prior retrieval-flavored or few-shot method relaxes at least one of these constraints. We emphasize that the novelty of our work lies in this unification, solving the common defects of prior literature through a scalable retrieval architecture.

3. Methodology

Fig. 2 illustrates the complete pipeline described in this section. The proposed system processes raw network traffic (stage 1 in Fig. 2) by organizing packets into bidirectional flows and extracting fixed-length feature vectors from the first N packets of each flow (stage 2). Two parallel learning branches then operate on these vectors (stage 3): (1) A HC branch that optionally re-weights features via a learned per-feature scaling vector, the W-vector $\mathbf{w}$; (2) A deep learning branch that trains a CNN or MLP backbone to map packet sequences to compact embeddings.

Both branches feed a unified CBR classification stage (stage 4), where representations are indexed in an HNSW ANN structure, and each query is classified by retrieving its closest stored neighbor. This same retrieval step enables OOD detection (stage 5) by treating flows whose nearest-neighbor distance exceeds a global threshold as unknown. To support early-classification constraints and real-world deployment scenarios where classification must occur with minimal observation, we restrict each flow to the first N packets. We evaluate two feature pipelines (HC feature pipeline, deep learning backbone pipeline) and two downstream classifiers (CBR, RF), which together define the main comparison axes of this study.

A core premise of our research is that effective classification can be achieved using a uniform, protocol-agnostic feature set derived solely from the first N packets of a flow. We intentionally avoid protocol-specific parsing (such as extracting TLS Handshake extensions) to maintain robustness against encryption updates (e.g., ECH in TLS 1.3).

For each flow, defined by its 5-tuple, we extract two types of features that together constitute the input for our HC pipeline, as summarized in Table 2:

1. **Statistical features (contextual behavior):** Aggregated metrics including minimum, maximum, mean, and standard deviation of packet sizes, inter-arrival times, and direction, calculated bi-directionally.
2. **Sequential features (raw fingerprint):** The ordered sequence of signed packet sizes $(S_1, S_2, \ldots, S_N)$, which preserves the shape of the handshake and initial data burst, and is critical for early identification.

By concatenating these two types of features, we construct a unified, protocol-agnostic feature vector of dimensionality $17 + N$. This vector serves as the foundational representation evaluated within our HC framework.

Table 1
Positioning of our retrieval framework against prior retrieval-flavored, few-shot, and class-incremental methods for encrypted traffic classification. The properties are: (a) a new class is added with no gradient steps, (b) no episodic meta-training or specialized pre-training stage is required before the encoder can be frozen, (c) no access to base-class data is needed when adapting, (d) no per-class prototype or centroid is recomputed, (e) OOD detection is performed within the same retrieval step, (f) search is backed by a scalable ANN index, and (g) the number of labeled samples available per new class is an input supplied at query time rather than a hyperparameter fixed before training. A check mark indicates the property holds, and a blank indicates it does not hold or is not addressed by that work. Where a work does not report a property, we resolve the cell in its favor. To the best of our knowledge, ours is the only framework that satisfies all seven simultaneously.

Method	(a)	(b)	(c)	(d)	(e)	(f)	(g)
Ma et al. [17] (weighted k-NN)	✓	✓	✓	✓			✓
Bovenzi et al. [22] (meta/transfer)	✓		✓				
Li et al. [26] (CL-MetaFlow)	✓		✓				
Zhao et al. [27]				✓			✓
Cai et al. [30] (CPN, incremental)		✓					✓
ILETC [31] (generative replay)		✓		✓			✓
Yang et al. [28] (MetaMRE)			✓	✓			
Di Monda et al. [29] (Meta Mimetic)	✓		✓	✓			
Ours (CBR)	✓	✓	✓	✓	✓	✓	✓

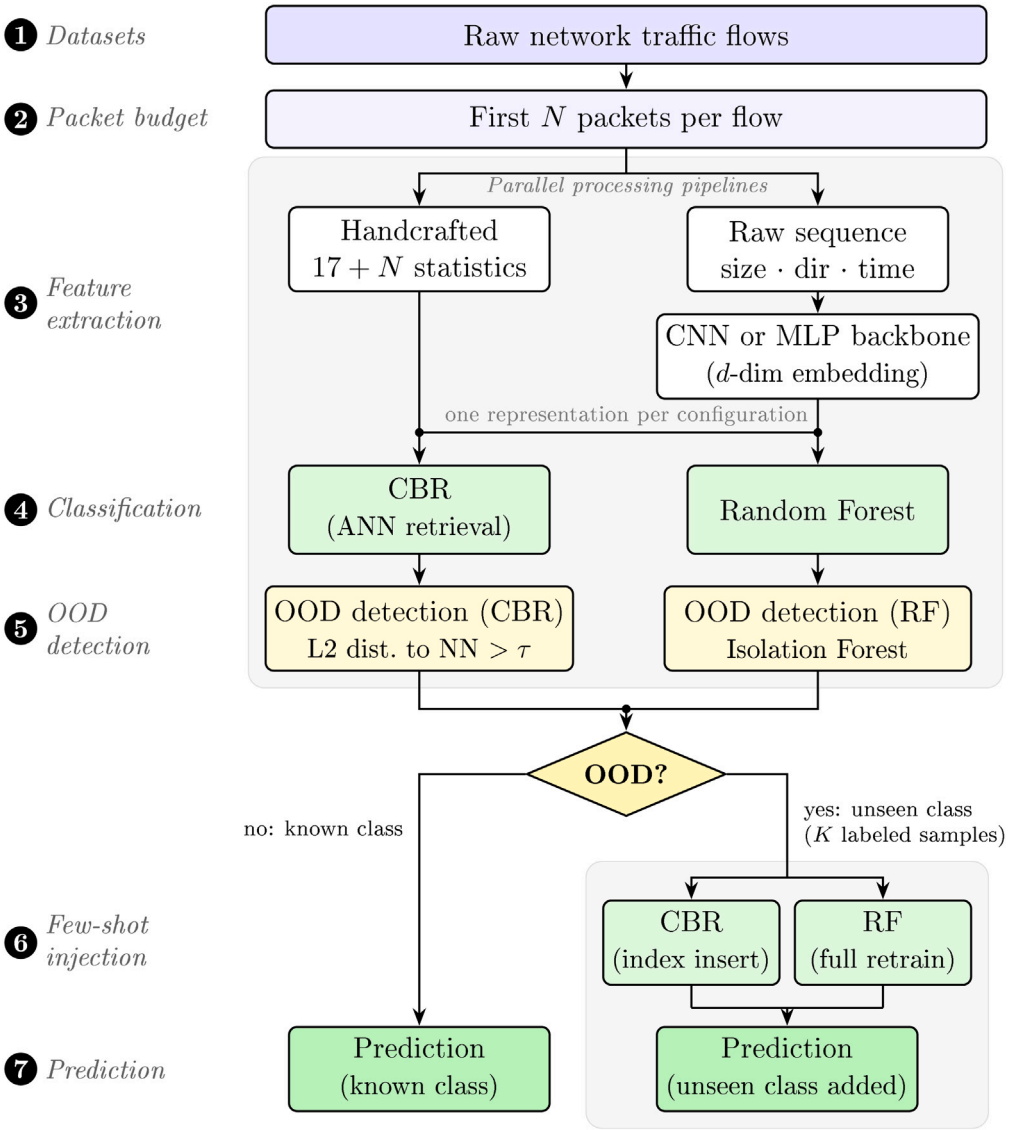


Fig. 2. Overview of the experiment pipeline. Badges in the left margin mark the stages referenced in the text. The routing bar above stage 4 pairs each representation with each classifier, which gives the configurations named in the text such as CNN-CBR and HC-RF, and does not combine the two representations. Both detectors feed one decision node, and flows flagged as OOD are routed to the few-shot stage. CBR is Classification By Retrieval over an HNSW index, and RF OOD uses Isolation Forest [34].

Table 2
Feature groups extracted per flow. Total dimensionality is $17+N$ (9 inter-arrival + 4 size statistics + 2 size delta + 2 PPS + N signed packet sizes).

Feature group	Slots in vector	Description
Packet sizes	N	Signed packet sizes (positive = client→server, negative = server→client)
Statistics of packet sizes	4	Min, max, mean, std of packet sizes
Size delta between packets	2	Mean and std of inter-packet size differences
Packets per second	2	Packets per second in each direction (forward and backward)
Inter-arrival time	9	Min, max, and mean of bidirectional, forward, and backward inter-arrival times

Handcrafted feature pipeline. This pipeline directly operates on the composite vector described above, which combines the 17 statistical metrics with the N sequential packet sizes (yielding a total of $17 + N$ dimensions; see Table 2). For the RF classifier, due to the fact that RF is inherently scale-invariant because tree splits are invariant to monotone per-feature transformations, it is trained on raw feature values with no normalization.

For the CBR classifier, we evaluate two normalization variants to process this combined vector:

- ANN-raw: Raw feature values are indexed directly into the HNSW structure without any preprocessing.
- ANN-norm (W-vector): A *LearnableScaler*, the per-feature scaling vector $\mathbf{w}$ that holds a single learned weight per feature, is trained with batch-hard triplet loss [35] to produce a class-discriminative re-weighting of the feature space. The scaled vector is then indexed. Unlike a deep backbone, this variant does not reduce dimensionality, but rather re-weights the existing $17 + N$ feature dimensions.

Deep learning backbone pipeline. In contrast to the HC approach that relies on explicit vector concatenation, the deep learning branch bypasses manual feature engineering by learning representations directly from raw network attributes. A CNN or MLP backbone is trained on the raw per-packet sequence and maps each flow to a compact d-dimensional embedding.

Both architectures receive the same $3 \times N$ input matrix whose rows correspond to per-packet inter-arrival time, direction, and size. Because these three channels differ by several orders of magnitude, a per-channel batch normalization layer normalizes the input before any learned transformation. The backbone is trained separately for each packet budget N and is frozen permanently once training is complete, with only its weights retained for embedding extraction.

CNN backbone. The CNN is a stack of four one-dimensional residual blocks with dilations $[1, 2, 4, 1]$ and channel widths progressing as $3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 128$. Each residual block applies a dilated convolution (kernel size 5) followed by a non-dilated convolution (kernel size 3), GELU activations, and batch normalization, with a learned 1×1 projection on the skip path when the channel count changes. Dilation progressively expands the receptive field across the packet sequence without increasing parameter count. After the four blocks, adaptive average pooling collapses the temporal dimension to a single vector, which a linear layer then projects to d dimensions. The output is L2-normalized to lie on the unit hypersphere.

MLP backbone. The MLP flattens the $3\times N$ input to a $3N$-dimensional vector and applies batch normalization at the input. Three hidden layers of widths 128, 64, and 32 follow, each consisting of a linear projection, ReLU activation, layer normalization, and dropout. Layer normalization is preferred over batch normalization in the hidden layers because the balanced batch sampler (described below) varies per-batch class composition across epochs, making batch statistics unreliable within hidden layers. A final linear layer projects to d dimensions, and the output is L2-normalized.

Shared training procedure. Both backbones are trained with the same procedure (stage 4). A cosine classification head with a learnable temperature scalar s (initialized to 10) computes logits as $s \cdot (\mathbf{z} \cdot \hat{W}^{\top})$, where $\mathbf{z}$ is the L2-normalized embedding and $\hat{W}$ is the row-normalized weight matrix of the classification head, distinct from the HC scaling vector $\mathbf{w}$ defined above. Training minimizes a joint loss combining cross-entropy and Supervised Contrastive loss [36] with a tuned SupCon weight temperature 0.1. Mini-batches are constructed with a balanced batch sampler drawing $P = 32$ classes with $M = 4$ samples each, yielding an effective batch size of 128 and ensuring every batch contains sufficient within-class pairs for the contrastive term. The optimizer is AdamW with learning rate 5×10^{-4} and weight decay 10^{-4}. The learning rate follows a three-epoch linear warmup followed by cosine annealing for the remaining epochs. Training stops early when validation accuracy does not improve for 10 consecutive epochs, and the checkpoint with the highest validation accuracy is retained. After training, the classifier head is discarded and the backbone weights are frozen for all downstream use.

Downstream classifiers. Both pipelines are evaluated with two classifiers: RF, a standard ensemble classifier trained on labeled feature vectors, used as a supervised baseline. We select RF for this role because prior encrypted-traffic studies establish it as a strong and often superior alternative to deep models while remaining efficient and robust to class imbalance [18–21], which makes it a demanding rather than a weak baseline. The second is CBR, which implements ANN search over an HNSW vector index (Elasticsearch [37]). CBR performs no discriminative training at inference time: every query is answered by finding its closest match in the index. We use the term CBR throughout the paper to emphasize this retrieval-only property, which serves as the primary focus of our efficiency and adaptability analysis. For consistency, we adopt a standard naming convention for all evaluated configurations, combining the feature backbone and the downstream classifier (e.g., CNN-CBR, MLP-CBR, and HC-RF).

Normalization is applied selectively based on the classifier. RF uses raw feature values in all cases: normalization does not improve tree-based classifiers, and omitting it ensures that normalization overhead is not charged against RF in runtime comparisons. For ANN on HC features, two regimes are compared: ANN-raw (no preprocessing) and ANN-norm (W-vector learned scaling, described above). Deep-backbone training requires no additional preprocessing beyond assembling the $3 \times N$ per-packet matrix; a separate backbone model is trained for each packet budget N.

3.1. Classification By Retrieval - CBR

CBR classifies each query vector by identifying its closest counterparts within a pre-indexed set of labeled reference vectors. Unlike parametric classifiers (e.g., RF or traditional neural networks) that require computationally expensive retraining cycles to accommodate

unseen classes, CBR bypasses inference-time model updating entirely. This retrieval step is realized through ANN search via the HNSW algorithm [7], which keeps it scalable by achieving $\mathcal{O}(\log n)$ query time. By leveraging an ANN architecture, adding a new class to the system simplifies to a pure indexing operation that injects new labeled vectors into the existing space, allowing for zero-downtime, few-shot adaptation.

HNSW builds a multi-layered proximity graph over the indexed vectors. During search, traversal begins at the coarsest layer and progressively narrows toward the nearest neighbor, achieving $\mathcal{O}(\log n)$ average query time without exhaustive comparison. The graph topology is controlled by connectivity parameter m and construction beam width $ef_construction$, which govern the index quality–build cost trade-off. The specific values used in our evaluation are given in Section 4.

Classification and anomaly detection are unified in this framework. When a query vector is submitted, its nearest indexed neighbor determines the predicted label. If the distance to that neighbor exceeds a threshold τ, the sample is flagged as OOD (see Section 3.3). Adding a new class requires only indexing its representative vectors, with no retraining or model recalibration needed, making the system directly suitable for few-shot class injection.

3.2. Few-shot adaptation pipeline

Each type of training follows the same staged pipeline, illustrated with numbered stages 1–7 in Fig. 2:

1. **Backbone training (optional):** Train a feature extractor on a set of base (seen) classes. After training, the backbone is frozen and used to generate embeddings for indexing and adaptation.
2. **Indexing:** Build reference representations for the currently known classes. In embedding-based variants, references are built in embedding space; in classic variants, references may be built directly in feature space.
3. **Few-shot class injection:** Introduce K labeled samples per unseen class into the system, which are absorbed by the few-shot classifiers (stage 6 in Fig. 2).
4. **Adaptation to unseen classes:** A primary comparison axis of this work is *how each classifier incorporates unseen classes* given only K labeled examples.
   - **CBR:** The K few-shot vectors are inserted directly into the existing HNSW index. No model parameters are updated; the new class becomes queryable immediately at $\mathcal{O}(K)$ indexing cost.
   - **RF:** The model must be retrained from scratch on the combined dataset (all base training samples ∪ the K few-shot samples per unseen class). Decision trees cannot be incrementally updated without a full structural rebuild.

   This asymmetry is central to our argument: in dynamic network environments where new application or malware classes emerge continuously, ANN can absorb new classes at negligible cost, while RF requires full retraining proportional to the entire training set size.
5. **Evaluation:** Evaluate classification outcomes for known classes (true vs. false predictions; stage 7 in Fig. 2). When enabled, also evaluate OOD detection outcomes.

3.3. Out-of-distribution detection

ANN-based OOD. OOD detection in the ANN pipeline is a natural byproduct of the ANN retrieval step. A query is flagged as OOD if its ℓ_2 distance to the nearest indexed neighbor exceeds a global threshold τ:

$$\mathrm{OOD}_{\mathrm{ANN}}(x) = \mathbb{I}[\,\rho(x) > \tau\,], \qquad \rho(x) = \min_{c \in C} \left\| \mathbf{z}(x) - r_c \right\|_2, \tag{1}$$

where x is the query flow, $\mathbf{z}(x)$ is its feature (or embedding) representation, C is the set of indexed classes, r_c is the nearest indexed vector of class c, $\|\cdot\|_2$ denotes the ℓ_2 distance between representations, $\rho(x)$ is the resulting distance to the nearest indexed neighbor, and $\mathbb{I}[\cdot]$ is the indicator function returning 1 when its condition holds and 0 otherwise. No additional model or computation is required beyond the search already performed for classification.

RF-based OOD. RF does not produce distances in a metric space, so a distance-based rule cannot be applied directly. Instead, we use **Isolation Forest** [34] as a standalone anomaly detector alongside the RF classifier. Isolation Forest builds an ensemble of random decision trees that recursively partition the feature space with random cuts, and data points that are isolated in fewer splits, because they lie in sparse, atypical regions, receive a high anomaly score and are flagged as OOD. The detector is trained once on the base training set (in-distribution data) and applied at test time independently of the RF classifier. Isolation Forest is well-suited to this setting: it is non-parametric, makes no distributional assumptions, and scales efficiently to the high-dimensional HC feature vectors used in our pipeline. We include Isolation Forest as a baseline to evaluate whether the retrieval step effectively detects OOD traffic compared to a standard anomaly detector under the same classification framework. Rather than competing with purpose-built OOD detection literature, our focus is demonstrating that effective OOD detection occurs naturally during the retrieval step required for classification, providing a zero-additional-cost mechanism without requiring a dedicated secondary model.

4. Experimental design

We evaluate few-shot adaptation for encrypted-traffic classification across multiple datasets with different properties and label spaces. Unless noted otherwise, each dataset is treated as a multi-class classification problem using the most relevant label level for that dataset.

4.1. Datasets

We conduct the classification and few-shot experiments on three datasets, each representing a distinct network environment. Each is used separately, and no model is ever trained on more than one dataset. This diversity demonstrates that our feature set is agnostic to the specific application type or protocol version. A fourth dataset, introduced below, is used only to probe OOD detection and is never trained on. While traditional benchmark datasets predominantly rely on TLS Server Name Indication (SNI) for ground-truth labeling, the advancing encryption landscape, driven by the adoption of ECH, is shifting label acquisition toward multi-source fallback cascades. Operational labeling pipelines increasingly infer ground truth from resolved DNS records, IP-to-service databases, and application-layer metadata available at the collection point. Crucially, label acquisition is an offline dataset-construction process and is strictly independent of the features consumed by our classifier, none of which are extracted from the TLS handshake.

- **CESNET-TLS22** [38] is a large-scale benchmark for encrypted traffic classification collected from the CESNET2 optical backbone. The full collection spans approximately two weeks and contains roughly 141 million flows across 191 application classes. The dataset combines standard flow statistics with detailed packet metadata for the first 30 packets. In our experiments we use a single day of traffic from this collection. After excluding classes with fewer than 100 samples, our subset comprises 184 application classes across 21 service categories, totaling 15,200,175 flows. This dataset was selected in part for how it was collected: the traffic is the production traffic of a national research and education backbone serving hundreds of thousands of users, captured on live links rather than generated in a controlled testbed. It therefore

Table 3
Aggregate statistics of the RADCOM cellular-traffic dataset. The capture consists of 11 short sessions recorded at separate, randomly chosen times, so that the traffic samples a variety of network conditions rather than one continuous window. The raw capture cannot be released due to subscriber-privacy and commercial-sensitivity constraints, so we publish the dataset-level aggregates permitted under the data-sharing agreement. No subscriber-identifying field (IP addresses, SNI values, session contents) is disclosed.

Statistic	Value
Capture sessions	11
Cumulative capture time	2.7 h
Total packets	542,827
Raw capture volume (pcap)	5.9 GB
Total payload volume	0.22 GB
Uplink packet share	47.1%
TCP/UDP packet share	81.2%/18.8%
Mean packet payload size	405 ± 539 B
Flows (before class filter)	17,147
Application labels (before filter)	246
Flows (≥100 samples per class)	13,893
Application classes (≥100 samples)	35
Flows per class (min/median/max)	105/287/1101
Packets per flow (median)	28
Flow duration (median)	2.0 s

inherently contains the noise of an operational network, including background and machine-generated services, short-lived and atypical flows, and heavy class imbalance. Beyond the class-frequency threshold above, we apply no additional cleaning to the flows. The per-class distribution is shown in Fig. 3. The few-shot classes used to evaluate OOD adaptation are listed in Table 4a.

- **Cellular Network Traffic (RADCOM):** A proprietary dataset of encrypted cellular network traffic provided by RADCOM Ltd. [39], a network analytics company that specializes in mobile network analysis. The data was collected from a live production cellular network under operational conditions, capturing real subscriber sessions. Obtaining this data was a deliberate choice rather than a convenience: we partnered with an industry provider specifically so that the evaluation would include traffic from an active operational network, with the anomalies, background services, and irregular flow behavior that laboratory captures do not reproduce. The dataset covers a diverse range of mobile application categories. Due to subscriber privacy constraints and commercial sensitivity, this dataset is not publicly available. Its use in this study was conducted under a data-sharing agreement, with all traffic anonymized at the point of collection. While this limits reproducibility on this specific dataset, it is also what the dataset contributes to the evaluation: results on a commercially sensitive production network demonstrate that the framework transfers to the deployment conditions it targets, which no public benchmark can fully emulate. Table 3 reports the aggregate characteristics of the capture that the agreement permits us to release. The long tail of its raw label space, 246 distinct application labels of which 211 fall below 100 flows, further reflects its operational origin. After excluding classes with fewer than 100 samples, the dataset comprises 35 application categories and 13,893 flows in total, with per-class counts ranging from 105 to 1101. Of these 35 classes, 13 are held out as unseen classes for few-shot evaluation and 22 are candidates for the main retrieval index. The retrieval-index experiments apply a stricter threshold of 300 samples, which excludes 6 of these candidates (each with 217 to 290 samples), so the base retrieval index comprises 16 application classes. The class distribution is shown in Fig. 4. The few-shot classes are listed in Table 4b.
- **MIRAGE-2019** [23] is a publicly available mobile-app traffic dataset collected at the University of Napoli Federico II between 2017 and 2019 by 300 volunteers exercising typical app use cases. The original dataset contains traffic of 40 Android apps, where each biflow is labeled with its Android-package name and the per-app biflow count ranges from 361 to 8246. We use the public release, in which we observe 47 distinct Android packages. After removing background and system packages and apps with fewer than 100 biflows, our usable subset comprises 23 application classes totaling 84,565 biflows. Following the 8-way few-shot protocol of Bovenzi et al. [22], we hold out 8 apps as unseen classes (22,851 biflows, per-class counts from 1213 to 6155) and keep the remaining 15 apps as the base classes (61,714 biflows, per-class counts from 254 to 9737), which we split 80/20 into 49,371 training and 12,343 test biflows.[1] The per-class distribution is shown in Fig. 5. The few-shot classes are listed in Table 4c.

Alongside these three datasets we use a fourth, **MTA (Malware Traffic Analysis)** [40], in a fundamentally different role. It is never used for training, classification, or few-shot adaptation. It serves only as an OOD probe of a kind the three application datasets cannot provide, namely genuinely malicious traffic rather than a benign application withheld from the label set. The captures are sourced from the public Malware-Traffic-Analysis.net repository and filtered to command-and-control and malware flows using the per-capture indicators of compromise published with each sample, yielding roughly 29,800 malware flows spanning seven families (Emotet, Hancitor, Qakbot, IcedID, Valak, Dridex, and Zloader). None of these families, and no malware at all, appears in the CESNET base classes. We present these flows to the CESNET index and the CESNET-trained RF and ask whether they are flagged as OOD, a cross-dataset transfer test described in Section 5.5 and motivated by the operational need to recognize unknown malicious traffic. Because MTA plays no part in the classification or few-shot experiments, it has no base class list or few-shot split.

4.2. Experimental parameters

All experiments are run across the following parameter grids.

Packet budget: Each flow is truncated to the first N packets, with $N \in \{5, 10, 15, 20, 25, 30\}$. All experiments are repeated at each budget to quantify the impact of limited observation length.

Few-shot budget: Unseen classes are introduced with up to K labeled samples each, with:

$$K \in \{5, 10, 20, 30, 50, 100\}. \tag{2}$$

Sampling is intended to be close to uniform across unseen classes, subject to availability.

A class can participate in a K-shot experiment only if it contains at least K labeled examples. As K increases, fewer classes meet this requirement, making direct comparison across K values difficult. To address this, we evaluate two complementary regimes. In the **expanding-class regime**, each K-shot experiment uses all few-shot classes with at least K samples, so the evaluated class set grows as K decreases. In the **fixed-class regime**, all K values are evaluated on the same subset, namely classes with at least 100 samples (the maximum K in our grid), ensuring results are directly comparable across K. Table

[1] The public release of MIRAGE-2019 exposes a subset of the original capture, and the per-group app assignments and code of Bovenzi et al. [22] were not released. We therefore adopt an analogous partitioning protocol rather than an exact reproduction. In the public subset the original few-shot band of apps with fewer than 1000 samples is populated mostly by background and system packages, so we instead match their 8-way few-shot setup by holding out the 8 least-populous usable apps, which all retain at least 254 samples and thus support the full K-shot grid. The tail we remove is 24 packages totaling only 504 biflows, all but one of them below the 100-biflow threshold, and it is dominated by Google Play services and other Android system processes rather than by user-facing apps.

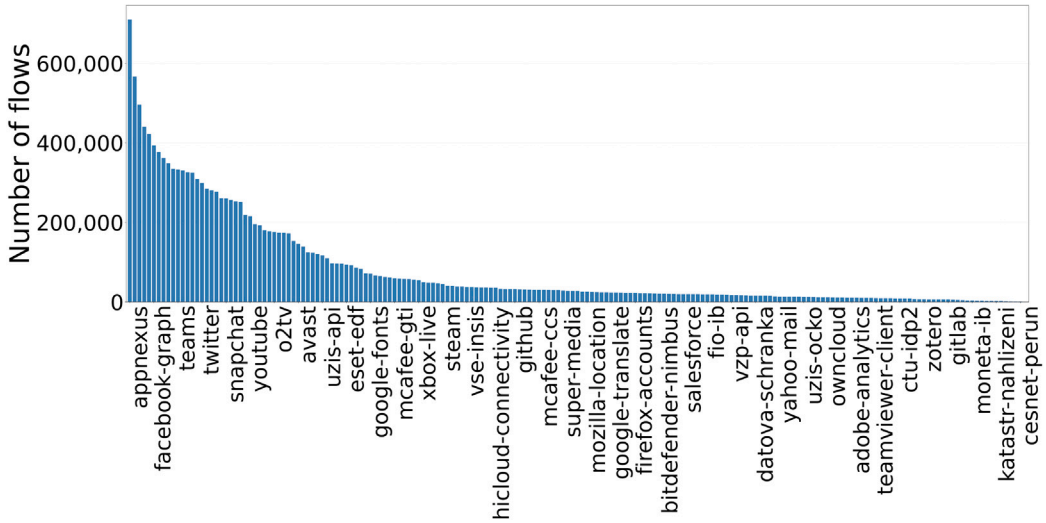


Fig. 3. Class distribution across all splits of the CESNET-TLS22 dataset (184 classes, which total 15,200,175 flows. The data is one day of traffic from the full two-week collection. Class names are printed for a regularly spaced subset of the bars, as 184 names cannot be set legibly side by side.

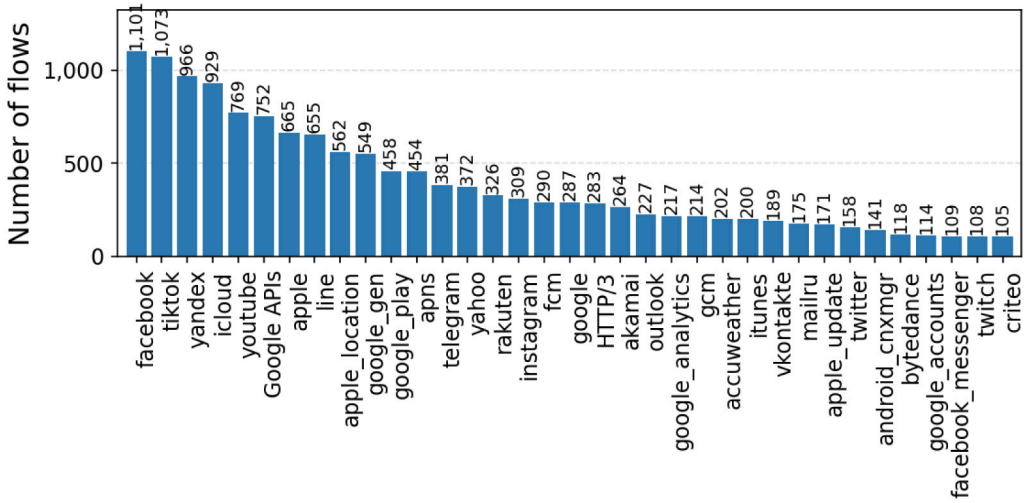


Fig. 4. Class distribution across all splits of the RADCOM dataset (35 classes shown: 22 main-set candidates plus the 13 few-shot classes, with classes below 100 samples excluded). After the 300-sample retrieval-index threshold, 16 of the 22 candidates form the base index.

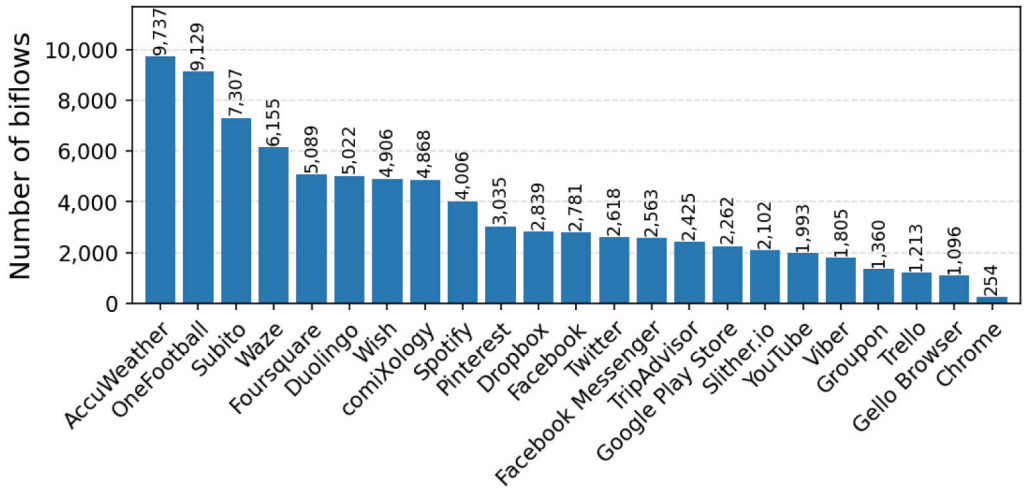


Fig. 5. Class distribution across all splits, MIRAGE-2019 dataset (23 usable classes, 84,565 total biflows, after excluding background and system packages and apps with fewer than 100 biflows).

Table 4
Few-shot classes per dataset ($K \in \{5, 10, 20, 30, 50, 100\}$). **Bold**: ≥100 samples, participates in all K-shot experiments. Non-bold: participates only when $K \leq$ sample count. The MIRAGE-2019 column counts biflows rather than flows, and all 8 of its held-out apps clear the threshold.

(a) CESNET-TLS22.		(b) RADCOM.		(c) MIRAGE-2019.	
Application	Flows	Application	Flows	Application	Biflows
eidentita	**920**	**gcm**	**214**	**Waze**	**6,155**
cesnet-perun	**742**	**accuweather**	**202**	**comiXology**	**4,868**
justice-isir	**169**	**itunes**	**200**	**Dropbox**	**2,839**
xiaomi-tracking	84	**vkontakte**	**189**	**Twitter**	**2,618**
adobe-search	81	**mailru**	**175**	**YouTube**	**1,993**
redmine	74	**apple_update**	**171**	**Viber**	**1,805**
ksrzis	12	**twitter**	**158**	**Groupon**	**1,360**
apple-siri	8	**android_cnxmgr**	**141**	**Trello**	**1,213**
sumava-camdata	6	**bytedance**	**118**	**Total**	**22,851**
Total	**2096**	**google_accounts**	**114**		
		facebook_messenger	**109**		
		twitch	**108**		
		criteo	**105**		
		Total	**2004**		

4a lists all few-shot classes with at least 5 samples (the minimum K); classes that participate in all experiments are shown in bold.

Embedding dimensions (Experiment 2): The deep backbone is trained to produce d-dimensional embeddings, with $d \in \{16, 32, 64, 128\}$. We trained backbones for all of these dimensions, but report results only for $d = 128$.

Elasticsearch index configuration: Vectors are stored as `dense_vector` fields in Elasticsearch, indexed with HNSW using $m = 16$ and $ef_construction = 100$, and ℓ_2-norm (`l2_norm`) similarity. At query time, `num_candidates` = 10 for standard classification and `num_candidates` = 100 for OOD detection, trading retrieval accuracy against latency. The score returned by Elasticsearch is related to the ℓ_2 distance ρ by $score = 1/(1+\rho^2)$, so the distance used for OOD thresholding is recovered as $\rho = \sqrt{(1/score) - 1}$.

OOD threshold evaluation: Rather than calibrating a single threshold value, ANN-based OOD detection is evaluated across a predefined grid of thresholds applied to the Elasticsearch score. The grid comprises 17 score thresholds:

$$\mathcal{T} = \{1.0,\ 0.5,\ 0.2,\ 0.1,\ 0.05,\ 0.005,\ 0.001,\ 8\times10^{-4},\ 7\times10^{-4},\ 6\times10^{-4},\ 5\times10^{-4},\ 4\times10^{-4},\ 3\times10^{-4},\ 2\times10^{-4},\ 10^{-4},\ 5\times10^{-5},\ 10^{-5}\}.$$

Because $score = 1/(1+\rho^2)$, each score threshold directly corresponds to a fixed ℓ_2 distance threshold τ. The Area Under the Receiver Operating Characteristic (ROC) curve (AUROC) is used as the primary OOD metric and is threshold-free. Per-threshold results characterize the system's operating points across the sensitivity–specificity trade-off.

4.3. ANN backend selection

Before the main experiments, we conducted a preliminary study to select the ANN backend that realizes the retrieval index. We compared several widely used ANN implementations, including FAISS, Annoy, and Elasticsearch, and found the differences in classification accuracy between them to be negligible.

Because accuracy does not meaningfully distinguish the backends, we select the backend on operational grounds. We adopt Elasticsearch because, unlike in-process libraries such as FAISS, Annoy, or hnswlib, it provides the systems properties our deployment requires: a persistent on-disk index that survives restarts without rebuilding, distributed and horizontally scalable indexing and search, and a REST interface for integration. Most importantly for our contribution, Elasticsearch supports real-time incremental insertion into a live, queryable HNSW index, whereas several of the alternatives (for example Annoy and the tree-based methods) must rebuild the index to admit new vectors. This property is what makes the $\mathcal{O}(K)$ few-shot class injection of Section 3.1 practical in an operational setting, since new classes become searchable immediately without reindexing the reference set.

4.4. Experiments

Experiment 1: HC features, CBR vs. RF. **Goal:** Establish a strong HC baseline and compare CBR and RF across all datasets in terms of classification accuracy, OOD detection, and few-shot adaptation speed. **Datasets:** CESNET-TLS22, RADCOM, and MIRAGE-2019. For each dataset and each packet budget N, we extract the $17 + N$ HC feature vector (Table 2). RF is trained on raw features, and CBR is evaluated in two configurations, ANN-raw and ANN-norm, as defined in Section 3. OOD detection is applied (distance threshold for ANN, Isolation Forest for RF) and few-shot class injection is performed with $K \in \{5, 10, 20, 30, 50, 100\}$ samples per unseen class.

Experiment 2: Deep backbone, in-domain evaluation. **Goal:** Evaluate whether a learned embedding backbone improves classification accuracy, OOD detection, and few-shot efficiency compared to the HC baseline (Experiment 1), using the same datasets and the same downstream pipeline. **Datasets:** CESNET-TLS22, RADCOM, and MIRAGE-2019. For each dataset, a CNN or MLP backbone is trained on a subset of base classes from that dataset using the $3 \times N$ raw-sequence input (size · direction · time), producing d-dimensional embeddings ($d \in \{16, 32, 64, 128\}$). A separate backbone is trained for each dataset–packet-budget combination ($N \in \{5, 10, 15, 20, 25, 30\}$). After training, each backbone is **frozen permanently**: it is never updated, fine-tuned, or modified at any subsequent stage of the experiment. The frozen backbone embeds all flows, and the downstream pipeline from Experiment 1 is then applied identically:

1. **Classification:** ANN retrieval over the HNSW embedding index, or RF trained on embeddings.
2. **OOD detection:** ANN uses the L2 distance threshold τ. RF uses Isolation Forest (contamination set to `auto`, letting scikit-learn estimate the inlier fraction from the training data)
3. **Few-shot injection:** Unseen classes are introduced with $K \in \{5, 10, 20, 30, 50, 100\}$ labeled samples. ANN indexes the new embeddings directly; RF retrains on the combined embedding set. The backbone remains frozen in both cases.

Results are compared directly against Experiment 1 to isolate the contribution of the learned embedding space over the HC features.

4.5. Metrics and efficiency

We report accuracy, macro-averaged precision, recall, and F1-score for all classification experiments. Because all datasets are class-imbalanced, we use macro averaging throughout: metrics are computed per class and then averaged equally across all classes, giving the same weight to rare and frequent classes alike. Where applicable, runtime

accounting includes normalization time (for ANN experiments) and can be decomposed by pipeline stage (feature extraction or embedding, indexing, retraining, inference).

Few-shot metrics. In the few-shot experiments, macro F1 and accuracy are both computed on the unseen-class test flows only, under OOD classification: the classifier is free to answer with any class it knows, base or unseen, and an unseen flow captured by a base class counts as an error for both metrics. The two metrics then summarize the same predictions differently. Macro F1 is averaged over the unseen classes only, so every new class carries equal weight regardless of how many flows it contributes, and the base classes contribute no per-class terms of their own. Accuracy is the fraction of unseen flows routed to their correct new class, weighting flows rather than classes. Accuracy sits visibly below macro F1 for a structural reason: because this test set contains only unseen-class flows, a base class can never claim a flow that belongs to it, so precision over the unseen classes is close to one by construction and lifts F1, whereas accuracy is driven entirely by how many flows reach the right class. Low absolute accuracy at small K is therefore expected, and the quantities the few-shot figures are read for, the comparison between CBR and RF and the trend in K, are unaffected by this gap.

Inference latency and memory footprint. Beyond training cost, we report what each method requires at serving time, since this is what a deployed classifier pays per flow. Inference latency is reported as milliseconds per flow, obtained by dividing the time to classify the full test set by the number of test flows. For CBR we report two figures. The end-to-end latency covers everything the client does per flow, namely vector construction, the query round-trip to Elasticsearch, and parsing the response. The retrieval latency is the search time Elasticsearch itself reports, which isolates the cost of the HNSW lookup from client-side and transport overhead. The former is what a deployment using a networked index would observe, and the latter is the cost of the retrieval step proper. RF latency is measured from an in-process `predict` call on the same test set, which is the most favorable accounting for the baseline. Memory footprint is the size of the artifact each method must keep resident to classify a flow: the serialized forest for RF, and the on-disk HNSW index for CBR. We report the serialized sizes, and we verified that they are faithful proxies for what each method actually occupies at serving time. For RF, the serialized file matches the combined size of the decision-tree arrays held in memory to within 5 percent (for example 1350 MB on disk against 1343 MB of arrays on CESNET at $N = 10$), and the resident set of a process holding the loaded model is never smaller than this. For CBR, Elasticsearch reports zero heap-resident segment memory for these indices, because Lucene memory-maps the HNSW graph and vector data and serves them from the page cache rather than the JVM heap, so the resident footprint of an index is its store size. The learned feature extractor is shared by both methods and is excluded, and it is small in any case, ranging from roughly 2 KB for the HC scaling vector to about 1 MB for a CNN backbone.

Hardware and software environment. All experiments were run on a single workstation to ensure that the reported runtimes are directly comparable across methods. The machine is equipped with two Intel Xeon Silver 4410Y CPUs (24 physical cores, 48 threads in total), 251 GB of RAM, and three NVIDIA RTX A4500 GPUs (20 GB each); deep-learning backbones were trained on a single GPU. The software stack is Ubuntu 22.04.5 LTS (Linux kernel 5.15), Python 3.11, PyTorch 2.6.0 with CUDA 12.4, scikit-learn 1.7.1 (Random Forest and Isolation Forest baselines), and Elasticsearch 9.1.0 for HNSW indexing and retrieval. Unless otherwise stated, all runtime measurements were obtained on this identical configuration.

5. Results

This section reports experimental results on the CESNET, RADCOM, and MIRAGE-2019 datasets, comparing CBR (Elastic HNSW) against RF across four evaluation dimensions: base classification performance, temporal dynamics, few-shot adaptation, and out-of-distribution detection. Throughout this section we discuss the primary performance metric, the macro-averaged F1-score, to ensure rigorous evaluation across the imbalanced class distributions of all three datasets. An exhaustive evaluation that additionally reports Accuracy and macro-averaged Precision and Recall is provided in the supplementary tables. All comprehensive tabular data, intermediate packet configurations, and complete metric profiles are available in our anonymized supplementary results repository [41]. The full source code required to reproduce these results will be released publicly upon acceptance. Training time t (seconds) measures the time to build the CBR index or fit the RF model. Feature extraction time is identical for both methods and is excluded from t. Results from both Experiment 1 (HC features; Section 4) and Experiment 2 (CNN and MLP backbones; Section 4) are reported jointly throughout each subsection. CESNET serves as the primary dataset for establishing and explaining results; RADCOM results validate robustness and highlight dataset-specific differences.

5.1. Base performance and training efficiency

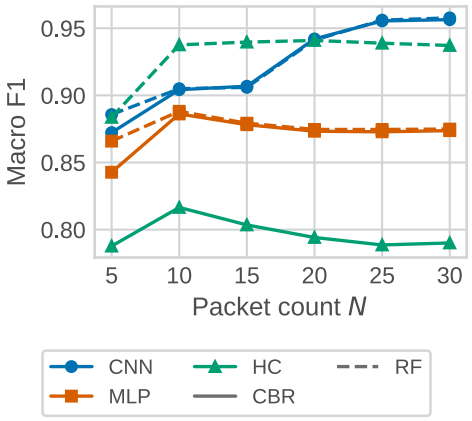
This subsection covers Experiments 1 and 2 jointly: HC features (Experiment 1) establish the baseline, and CNN/MLP embeddings (Experiment 2) are compared against it and against the RF baseline across all datasets. The evaluation on CESNET reveals a consistent pattern across all backbones: RF achieves higher classification accuracy than CBR, though the magnitude of this gap varies substantially with the choice of feature representation. Among all configurations, the CNN backbone delivers the strongest CBR performance, followed by MLP embeddings and then HC features, establishing a CNN > MLP > HC ranking for retrieval. This ordering does not carry over to RF, where HC features become the most competitive and MLP trails the other two backbones.

In terms of absolute accuracy, CNN-CBR and CNN-RF on CESNET produce nearly identical macro F1 scores, averaging 0.923 and 0.925 respectively across all packet-count configurations (Fig. 6(a)). This near-parity indicates that CBR operates at the frontier of what RF can achieve when using deep convolutional features, making the two approaches practically interchangeable in terms of predictive quality on this dataset. The MLP backbone is nearly on par between the two classifiers (0.871 for CBR vs. 0.876 for RF), while HC features exhibit the largest gap of all: HC-RF outperforms HC-CBR by an average margin of 0.133 F1 points. This pronounced gap for HC features reflects two compounding effects.

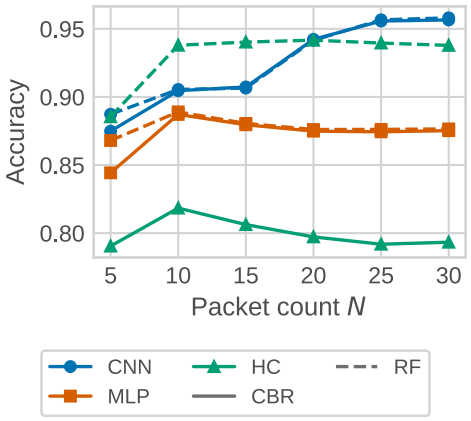
The raw feature geometry produced by HC statistics does not lend itself naturally to nearest-neighbor retrieval, whereas tree-based ensembles can construct more discriminative decision boundaries in that space. Furthermore, with `num_candidates` = 10, each CBR query explores only a small region of the index; in the HC feature space, where class boundaries overlap more, a small candidate pool increases the chance of returning a wrong nearest neighbor. In the deep-backbone pipelines, contrastive training clusters same-class embeddings tightly so that 10 candidates suffice for correct retrieval.

CBR is dramatically faster to train than RF across every backbone, as Fig. 7 makes visually explicit. On CESNET, CNN-CBR requires approximately 133 s versus 1270 s for CNN-RF, a roughly 10× speedup. For MLP embeddings, the advantage grows to nearly 50× (146 s vs. 7400 s), and for HC features CBR trains in 44 s against 1926 s for RF. The gap widens dramatically in the few-shot regime (Fig. 7(b)), where injecting $K = 5$ vectors into the index makes CBR two to three orders of magnitude faster than full RF retraining.

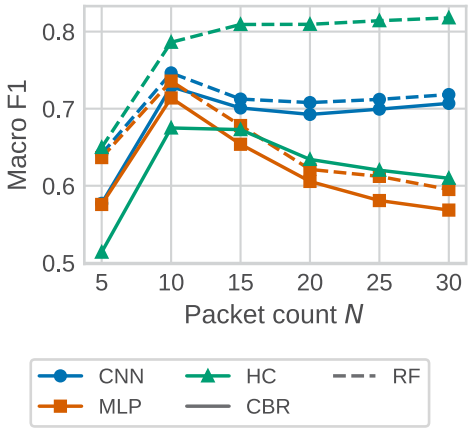
On MIRAGE-2019 the retrieval ranking is preserved (Fig. 6(e)): CNN-CBR and CNN-RF are again close (mean macro F1 0.772 vs. 0.787 across the six packet counts), MLP follows (0.714 vs. 0.733), and HC-RF remains the strongest HC configuration by a wide margin (0.824 vs. 0.651 for HC-CBR). CBR index construction stays one to two orders of



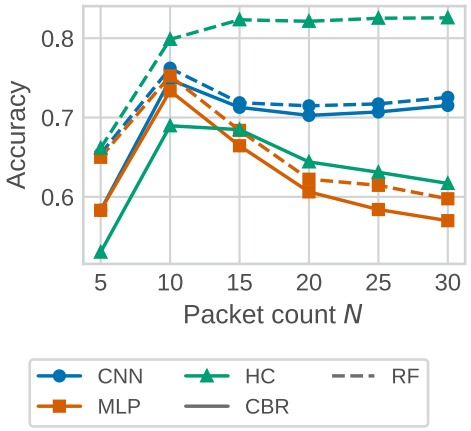
(a) CESNET (184 classes), macro F1



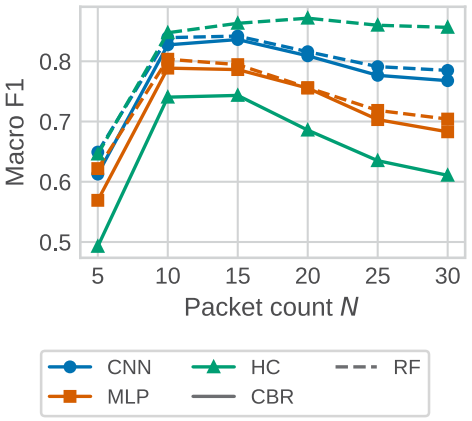
(b) CESNET (184 classes), accuracy



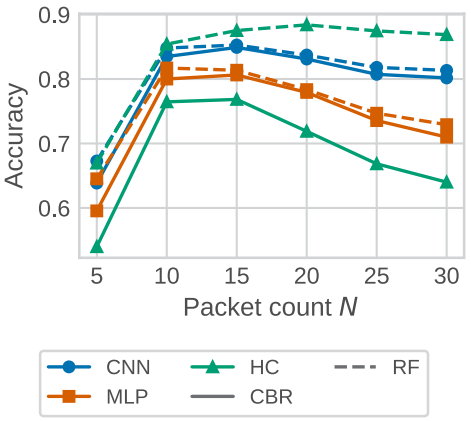
(c) RADCOM (16 classes), macro F1



(d) RADCOM (16 classes), accuracy

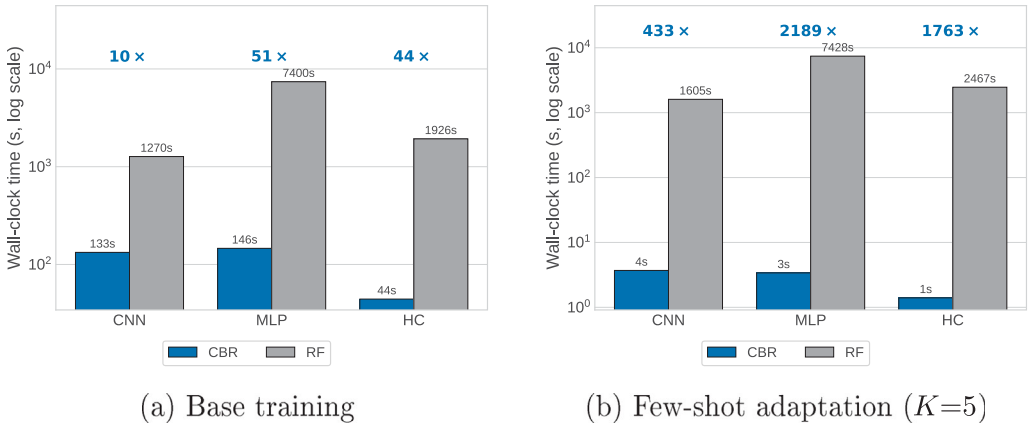


(e) MIRAGE-2019 (15 classes), macro F1



(f) MIRAGE-2019 (15 classes), accuracy

Fig. 6. Base classification performance as a function of the observed packet count N for CBR (solid) and RF (dashed) across the CNN, MLP, and HC backbones (color and marker per backbone). One row per dataset, with macro F1 in the left column and accuracy in the right. Unlike the few-shot setting, base accuracy is the standard test-set accuracy over all classes and tracks macro F1 closely. For CBR the CNN > MLP > HC ranking holds across packet counts, CNN-CBR and CNN-RF are near-identical on CESNET, and CBR trains roughly 10–50× faster than RF (training times reported in the text). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)



(a) Base training (b) Few-shot adaptation (K=5)

Fig. 7. Training and adaptation efficiency on CESNET application classification (184 classes), CBR (Elastic HNSW) versus RF, on a logarithmic time axis (mean over the six packet counts N). (a) base training time, where CBR is roughly 10× (CNN), 51× (MLP), and 44× (HC) faster than RF. (b) few-shot adaptation time at $K = 5$, where injecting new vectors into the index makes CBR two to three orders of magnitude faster than full RF retraining (433–2189×). The bold factor above each backbone group is the RF/CBR speedup.

Table 5
Inference latency and memory footprint, HC features, at $N = 10$ packets. Ranges in parentheses span all six packet budgets ($N \in \{5, \ldots, 30\}$). RF is faster per flow, while the CBR index is 5 to 38 times smaller than the serialized forest in every configuration. CBR retrieval is the search time Elasticsearch reports, and CBR end-to-end additionally includes vector construction and the query round-trip. Best value in each column is in bold.

(a) Memory footprint (MB).

Dataset	Classes	Indexed flows	RF model	CBR index
CESNET	184	1,198,995	1350 (1350–1586)	**171** (140–263)
RADCOM	16	8256	33 (29–36)	**1.1** (0.9–1.7)
MIRAGE	15	49,371	60 (60–75)	**7.0** (6.2–11.0)

(b) Inference latency (ms per flow).

Dataset	RF	CBR retrieval	CBR end-to-end
CESNET	**0.052**	0.147	0.289
RADCOM	**0.064**	0.328	0.757
MIRAGE	**0.017**	0.198	0.378

magnitude faster than RF fitting, mirroring the CESNET and RADCOM efficiency gap.

The RADCOM dataset (Fig. 6(c)) validates the accuracy and speed trends observed on CESNET, with one notable nuance. The HC feature gap is in fact slightly larger on RADCOM (0.160 average margin across packet counts), reinforcing the conclusion that HC features are structurally disadvantaged in a retrieval setting. Training efficiency is fully replicated: CBR consistently trains 6 to 15 times faster than RF depending on the backbone. However, the relationship between CNN-CBR and CNN-RF diverges quantitatively across the two datasets. On CESNET the two are essentially tied, whereas on RADCOM, RF leads CNN-CBR by 1 to 6 percentage points across packet-count configurations. This divergence suggests that CBR's competitiveness scales with dataset complexity: on a large and diverse dataset such as CESNET with 184 application classes, the deep embedding space is rich enough that retrieval-based classification matches discriminative learning, whereas on the smaller RADCOM 16-class subset the reduced task complexity gives RF more room to exploit its ensemble structure.

5.2. Inference latency and memory footprint

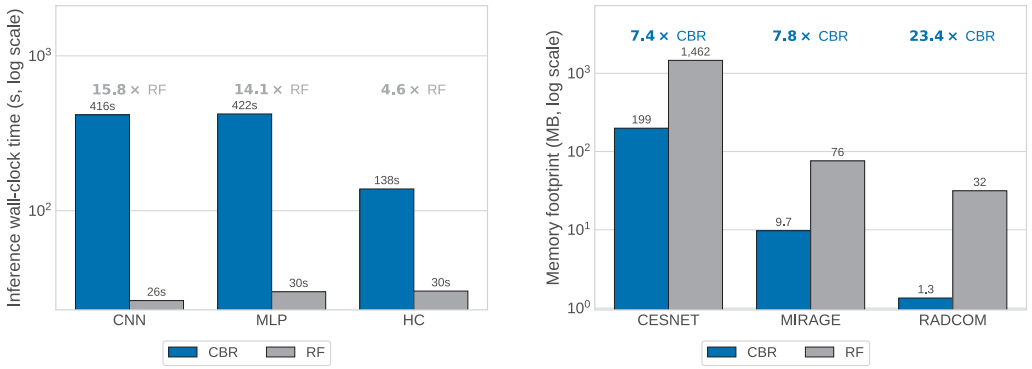
Training speed and adaptation speed are only part of the deployment cost. This subsection reports what each method pays at serving time (Fig. 8 and Table 5). The two methods trade places: RF classifies an individual flow faster, while CBR needs substantially less memory to do so. Latency is reported for every backbone, since the classification experiments recorded it. The footprint measurements were made on the HC pipeline across all three datasets and all six packet budgets, so the memory comparison is grouped by dataset.

Latency favors RF. A Random Forest resolves a flow in 0.017 to 0.068 ms, whereas CBR needs 0.13 to 0.15 ms on CESNET, 0.17 to 0.20 on MIRAGE and 0.32 to 0.33 on RADCOM for the retrieval step itself. Measured end to end, including vector construction and the query round-trip to Elasticsearch, CBR requires 0.24 to 0.83 ms per flow, so roughly half of the end-to-end cost is client-side and transport overhead rather than search. The honest reading is that a tree ensemble evaluated in process is the faster classifier per flow, and the size of that advantage depends strongly on the feature representation (Fig. 8(a)). On the CESNET test set, RF is 4.6× faster than CBR with HC features, but 15.8× faster with the CNN backbone and 14.1× faster with the MLP backbone. The asymmetry follows from where each method spends its time. A retrieval query must compute distances over the full embedding, so its cost grows with the width of the vector, from 22 to 47 HC dimensions up to 128 for the learned backbones. A forest of depth 15 instead traverses at most 15 comparisons per tree regardless of how wide the input is, which leaves RF inference almost unchanged across representations at 26 to 30 s for the whole test set. This does not undermine the practicality of retrieval. Even the slowest configuration sustains more than 1200 flows per second on a single unoptimized client, and the retrieval cost is remarkably insensitive to scale: CESNET holds 145 times more flows than RADCOM in its index, yet its per-flow search time is lower, which is the $\mathcal{O}(\log n)$ behavior of HNSW showing through.

Memory strongly favors CBR. The serialized forest is larger than the HNSW index in every configuration we measured, by a factor of 5 to 38. On CESNET the gap is the most consequential in absolute terms: the forest occupies 1350 to 1586 MB, while the index that answers the same 184-class query holds all 1,198,995 training flows in 140 to 263 MB.

5.3. Temporal dynamics

Across all datasets, the most substantial accuracy improvement occurs when the number of observed packets per flow increases from 5 to 10. On CESNET, CNN-CBR F1 rises from 0.872 at $N = 5$ to 0.904 at $N = 10$, an improvement of over 3 percentage points (Fig. 6(a)). A comparable leap appears on RADCOM, where CNN-CBR climbs from 0.577 to 0.729 over the same transition (Fig. 6(c)). This pattern reflects the fundamental informativeness of early packet payloads: a single initial packet is often insufficient to discriminate encrypted application traffic reliably, but capturing the first ten packets provides enough behavioral signal to characterize the flow with high confidence.



(a) Inference latency (b) Memory footprint

Fig. 8. What each method costs at serving time, on logarithmic axes (mean over the six packet counts N). The bold factor above each group is the advantage of the method that wins, colored to match it, and the two panels are won by different methods. (a) inference time for the CESNET test set (344,176 flows, 184 classes), where RF is the faster classifier by 4.6× with HC features but by 14 to 16× with the CNN and MLP backbones, because retrieval cost grows with the width of the embedding while a depth-limited forest is largely insensitive to it. (b) memory footprint of the HNSW index against the serialized Random Forest, where CBR is the smaller artifact on every dataset, by 7.4× on CESNET and by 23× on RADCOM. Panel (b) is grouped by dataset rather than by backbone because the footprint measurements were made on HC features. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Beyond $N = 10$, the two datasets diverge in their trajectories. On CESNET, accuracy continues to improve monotonically as the packet count rises from 10 to 30, with CNN-CBR reaching 0.956 at $N = 30$. This steady gain indicates that longer flows on CESNET carry progressively more discriminative content, rewarding models that observe more of the traffic sequence. On RADCOM, by contrast, performance plateaus after $N = 10$ and decreases slightly at higher packet counts in several configurations. This saturation likely reflects a structural property of RADCOM flows: mobile application traffic tends to produce shorter and more stereotyped exchanges, so packets beyond the initial burst introduce noise rather than additional signal. The finding implies that the optimal N should be selected based on the characteristics of the target network environment rather than set to a universal maximum.

The MLP backbone follows the same trend as CNN on CESNET and shows no degradation at short flow lengths. MLP-CBR reaches 0.843 at $N = 5$ and stays within 0.873 to 0.886 across the remaining packet counts, closely tracking MLP-RF (0.866 at $N = 5$). The CNN advantage over MLP therefore remains modest throughout, consistent with both backbones learning a metrically meaningful embedding even when only the first five packets are observed.

5.4. Few-shot adaptation

Each few-shot experiment injects K labeled samples per unseen class into a base model trained on the original class set. Macro F1 and accuracy are computed on the unseen-class test flows only, under the OOD protocol defined in Section 4.5: including base-class flows would dilute the score because the unseen classes contribute only a small fraction of the test set. The two metrics appear in the left and right columns of Figs. 9, 10 and 11 respectively. For CBR, injecting K samples is a constant-time index update with no retraining. For RF, the classifier is retrained from scratch on the base training set combined with the K new samples. Table 6 reports the few-shot macro F1 gain from $K = 5$ to $K = 100$ shots at $N = 10$ packets across the three datasets.

A consistent and prominent finding emerges: few-shot macro F1 (Figs. 9, 10, and 11) improves monotonically as K increases across all backbones and all three datasets. On CESNET with a CNN backbone and $N = 20$ packets, CBR F1 rises from 0.592 at $K = 5$ to 0.841 at $K = 100$, while RF F1 improves from 0.363 to 0.673 over the same range. Both methods benefit from richer support sets, but the rate and absolute level of improvement differ meaningfully between them.

For the CNN backbone, CBR consistently outperforms RF in the few-shot regime on CESNET, achieving on average a macro F1 of 0.725 against 0.572 for RF across all K and packet-count combinations. This advantage is intuitive: a retrieval system can immediately incorporate new examples into its case base without any retraining, making it structurally well-suited to few-shot extension. RF must rebuild its ensemble to accommodate unseen classes, and with few examples per unseen class its generalization is limited by the scarcity of the support. The speedup mirrors the base-training case: inserting K new vectors into an HNSW index requires $\mathcal{O}(K \log n)$ operations, whereas RF retraining from scratch is $\mathcal{O}((n+K) \cdot d \cdot T)$, where n is the number of training (indexed) flows, d is the feature dimension, and T is the number of trees. For MLP embeddings, CBR retains a smaller but consistent advantage on CESNET (0.655 vs. 0.582). HC features reverse the relationship: HC-RF achieves 0.723 compared to HC-CBR at 0.570, mirroring the base-performance pattern and reinforcing that the HC feature space is more effectively exploited by discriminative ensemble methods in both full-data and few-shot regimes.

Table 6
Few-shot adaptation at $N = 10$ packets, the early-classification operating point that also matches the $N_p = 10$ setting of Bovenzi et al. [22] (their N_p is the same packet budget as our N): macro F1 on the held-out classes shown as the gain from $K = 5$ to $K = 100$ shots, for CBR and RF across the CNN, MLP, and HC backbones. Macro F1 improves sharply with more shots, by up to roughly threefold. Bold marks the higher $K = 100$ value per backbone: CBR leads on the deep backbones for CESNET and MIRAGE, RF leads on HC throughout, and RF edges CBR across RADCOM, most narrowly for CNN (0.759 against 0.758). Where CBR and RF are close, CBR is still the better choice, because its adaptation is two to three orders of magnitude faster (Fig. 7).

Dataset	Method	CNN	MLP	HC
CESNET	CBR	**0.595 → 0.826**	**0.469 → 0.805**	0.481 → 0.750
	RF	0.409 → 0.777	0.429 → 0.793	**0.485 → 0.906**
RADCOM	CBR	0.332 → 0.758	0.323 → 0.711	0.334 → 0.733
	RF	**0.321 → 0.759**	**0.321 → 0.757**	**0.324 → 0.776**
MIRAGE	CBR	**0.383 → 0.751**	**0.309 → 0.683**	0.236 → 0.636
	RF	0.287 → 0.717	0.263 → 0.659	**0.249 → 0.696**

Two class-set variants are reported for CESNET. The *varying* variant includes all unseen classes with at least K samples, so the class count shrinks as K grows (9 classes at $K = 5$, 3 at $K = 100$). The *fixed* variant evaluates only the 3 unseen classes present at all K values.

5.4.1. CESNET application classification

Fig. 9 presents few-shot macro F1 and accuracy for the CNN, MLP, and HC pipelines under the varying class set. Across all backbones, CBR

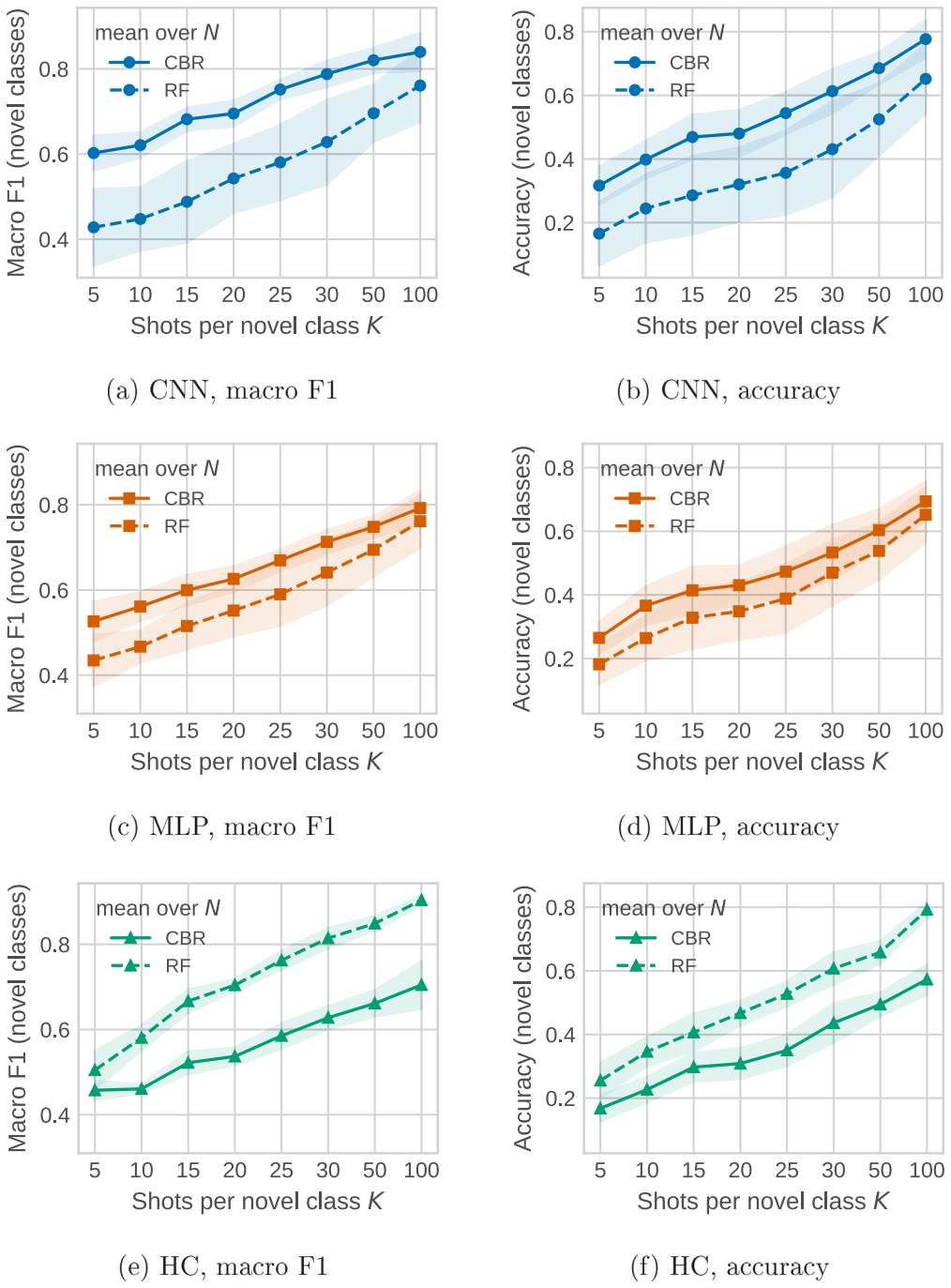


Fig. 9. CESNET application few-shot adaptation (varying class set) as a function of the per-class shot budget K, for CBR (index insertion only, solid) and RF (full retraining, dashed). One row per backbone, with macro F1 on the unseen classes in the left column and accuracy on the same flows in the right column. Each curve is the mean across the six packet counts N and the shaded band is one standard deviation. Both columns score the same OOD predictions, in which a base class may capture an unseen flow, and differ only in the summary: macro F1 averages per-class scores over the unseen classes, while accuracy is the fraction of unseen flows routed to their correct new class (Section 4.5), which is why the right column sits lower. Both metrics rise monotonically with K and rank the methods identically: CNN-CBR leads CNN-RF, MLP-CBR holds a smaller lead, and HC-RF reverses the relationship over HC-CBR. CBR adaptation takes 2–5 s versus 1000–2500 s for RF retraining.

adaptation requires only 2–5 s while RF retraining takes 1000–2500 s on CESNET, reflecting the fundamental complexity difference between HNSW index insertion and full ensemble retraining.

5.4.2. RADCOM application classification

The RADCOM dataset (Fig. 10) validates all three few-shot dynamics established on CESNET. CNN-CBR leads CNN-RF in few-shot classification (average F1 0.392 vs. 0.382 across all K and N), and HC-RF outperforms HC-CBR (0.531 vs. 0.435) across all N and K values. The absolute advantage for CNN-CBR is smaller on RADCOM than on CESNET, consistent with the lower overall complexity of the 16-class task. MLP few-shot performance is effectively tied between CBR and RF on RADCOM (0.385 vs. 0.390), compared to CBR's clearer advantage on CESNET, a minor divergence that reflects the reduced task complexity rather than a structural change in behavior. CBR adaptation requires

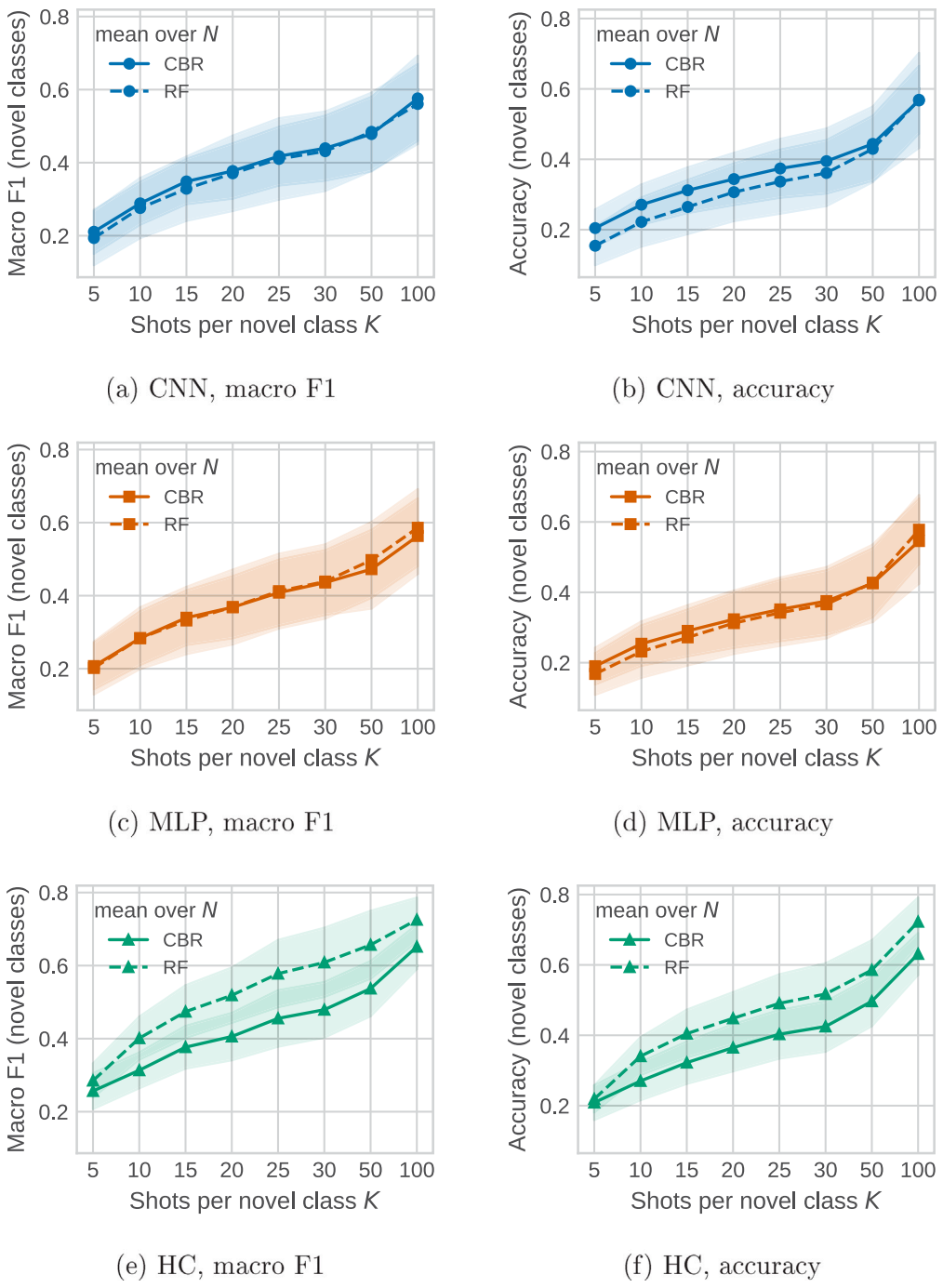


Fig. 10. RADCOM application few-shot adaptation (varying class set) as a function of the per-class shot budget K, for CBR (solid) and RF (dashed). One row per backbone, with macro F1 on the unseen classes in the left column and accuracy on the same flows in the right column. Each curve is the mean across the six packet counts N and the shaded band is one standard deviation. The trends mirror CESNET at lower absolute values: CNN-CBR edges CNN-RF, MLP is effectively tied, and HC-RF leads HC-CBR. The one place the two metrics disagree is CNN at $K = 100$, where CBR keeps a macro F1 lead but RF draws marginally ahead on accuracy (0.569 against 0.567). CBR adaptation takes 1–7 s versus 64–130 s for RF retraining.

1–7 s for deep backbones and 2 s for HC, while RF retraining requires 64–130 s.

5.4.3. MIRAGE-2019 application classification

The MIRAGE-2019 dataset (Fig. 11) lets us position our retrieval-based few-shot adaptation against the published meta-learning and transfer-learning results of Bovenzi et al. [22] on the same benchmark. MIRAGE-2019 reproduces the three few-shot dynamics established on CESNET and RADCOM. CNN-CBR leads CNN-RF (average macro F1 0.315 vs. 0.235 across all K and N), MLP-CBR leads MLP-RF (0.342 vs. 0.280), and HC-RF reverses the relationship over HC-CBR (0.351 vs. 0.311). Few-shot F1 rises monotonically with K for every backbone: at $N = 10$ packets CNN-CBR improves from 0.383 at $K = 5$ to 0.751 at $K = 100$, and MLP-CBR from 0.309 to 0.683. CBR adaptation requires 34–37 s for the deep backbones and 7–10 s for HC, while RF retraining requires 187–1310 s.

At the matched protocol, balanced 8-way classification among the held-out apps at $N_p = 10$, CNN-CBR reaches a macro F1 of 0.505 at $K = 5$ and 0.814 at $K = 100$, and MLP-CBR reaches 0.482 and 0.796. Fig. 12 places these against all eight of their methods over the

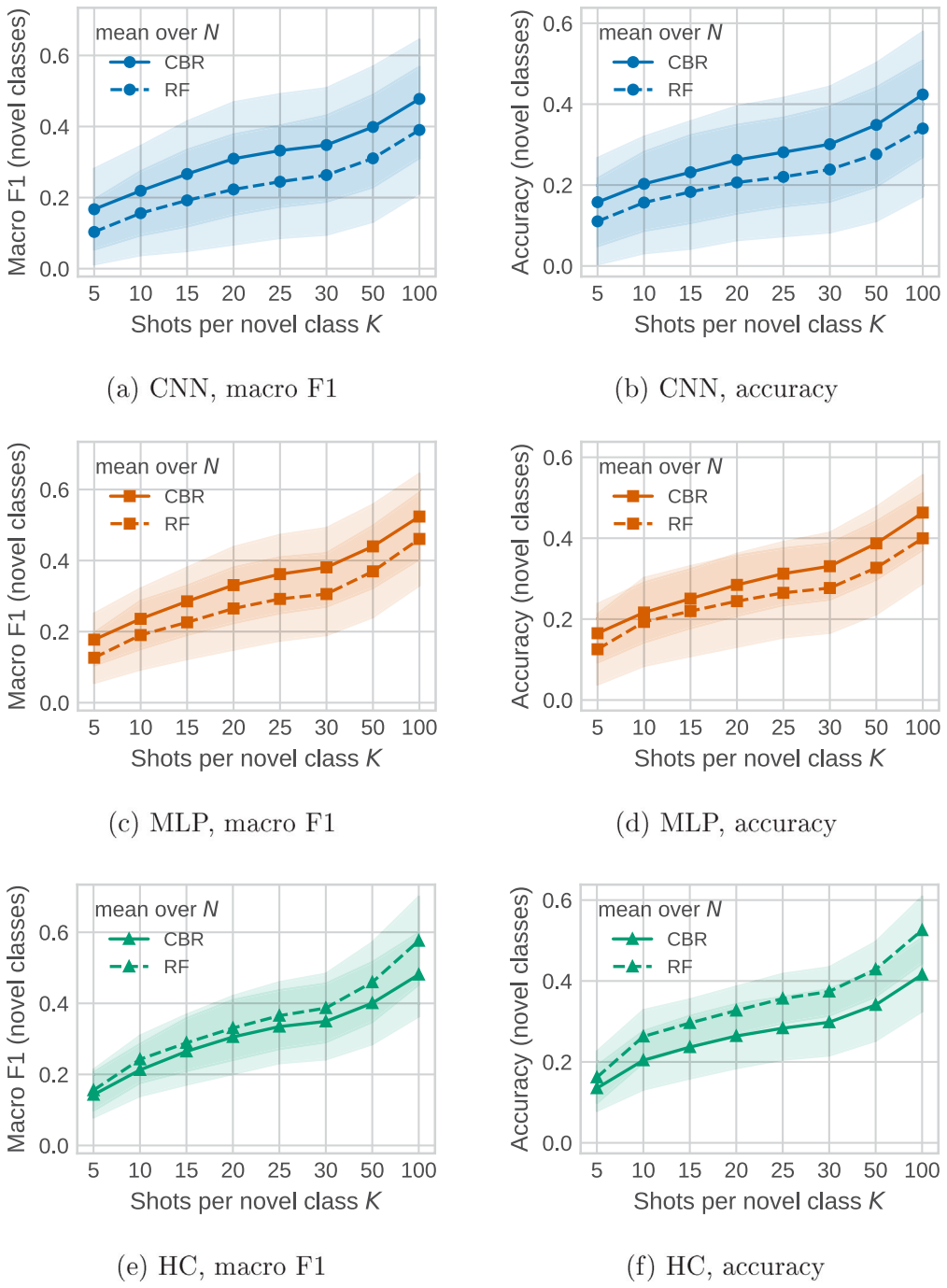


Fig. 11. MIRAGE-2019 application few-shot adaptation on the 8 held-out classes as a function of the per-class shot budget K, for CBR (index insertion only, solid) and RF (full retraining, dashed). One row per backbone, with macro F1 in the left column and accuracy on the same flows in the right column. Each curve is the mean across the six packet counts N and the shaded band is one standard deviation. Both metrics rise monotonically with K and agree on the ranking: as on CESNET and RADCOM, CNN-CBR and MLP-CBR lead their RF counterparts while HC-RF reverses the relationship over HC-CBR. CBR adaptation takes 4–20 s versus 70–491 s for RF retraining.

full shot range. MatchingNet [25] slightly outperform CNN-CBR across most shot regimes by a margin of at most 3.5 points, whereas both methods reach parity at the $K = 5$ evaluation. At $K = 5$ the two are level, 0.505 against the 50% their text reports. CNN-CBR is above TLFZ, TLFT and ProtoNet [24] at four of the five budgets and within 0.02 at the fifth, above MetaOptNet [42] at every budget they report, and above fo-MAML [43], ANIL [44] and RelationNet [45] throughout, none of which passes 0.61 at any budget. Retrieval therefore tracks the strongest of their meta- and transfer-learning methods to within the width of an episode, and leads all seven of the others at $K = 50$ and $K = 100$, without meta-training any model.

Two capabilities separate the two settings further, and neither is visible in the accuracy figure. The first is that no method in their study can decline to answer, whereas the same retrieval step that classifies a flow also yields its distance to the nearest indexed neighbor, which we threshold to flag an application the index has never seen (Section 5.5). The second is the input each method consumes. Their embedding function reads four per-packet fields, the transport-layer payload size,

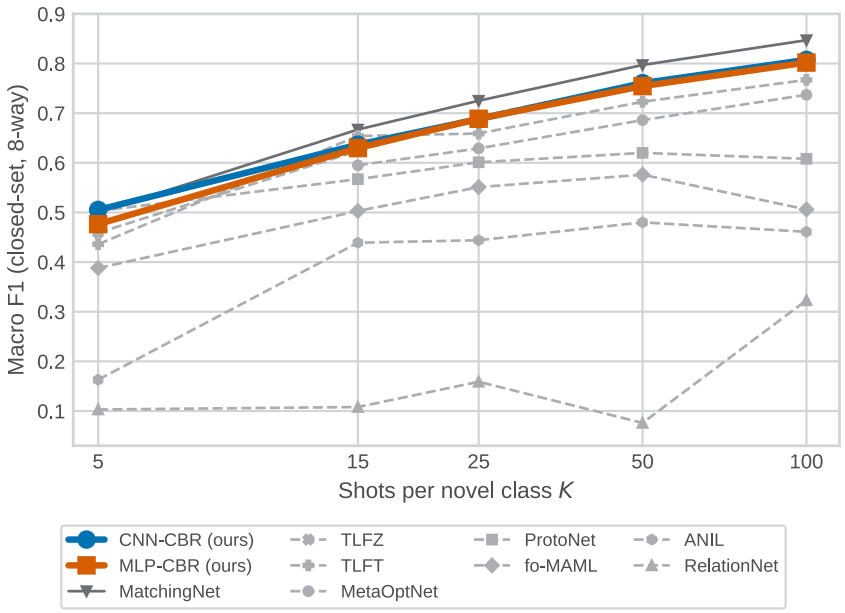


Fig. 12. MIRAGE-2019 closed-set comparison at the matched protocol of Bovenzi et al. [22]: balanced 8-way macro F1 among the held-out apps at $N_p = 10$ as a function of the shot budget K. Colored curves are our retrieval with the CNN and MLP backbones, each the mean over episodes. Grey curves are all eight methods of Bovenzi et al. over the same shot range, digitized from their Fig. 2 because no numeric results were released, with a reading uncertainty of about 0.02. Dark grey marks MatchingNet, which gain the same results as us at $K = 5$. Light grey marks the remaining seven, all of which CNN-CBR leads at $K = 50$ and $K = 100$. The two curves begin level, 0.505 against 0.500 at $K = 5$, and MatchingNet opens a margin of at most 3.5 points thereafter. Note that K carries a different cost on each side: for their methods it is an episode size fixed at meta-training, so every point is a separately meta-trained model, whereas for ours it is the number of vectors inserted into a fixed index. The two evaluations use different releases of the dataset, so the comparison is best-effort rather than controlled. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

the packet direction, the TCP window size, and the inter-arrival time. Ours reads three, the inter-arrival time, the direction and the packet size, and carries no transport-layer state, which makes it applicable to UDP and QUIC flows as well as TCP.

Their protocol also bounds what the comparison can show. Meta-testing draws balanced 8-way episodes with 100 queries per class, so chance accuracy is 12.5%, the correct application is guaranteed to be among the eight candidates, and the heavy class imbalance of MIRAGE-2019, whose applications range from 361 to 8246 biflows, is removed by construction. Our OOD results in Section 5.5, where a query may be rejected as unknown, have no counterpart in their work.

The remaining gap should also be read with two caveats. Bovenzi et al. report per-method scores only as figures, without a numeric results table or a public implementation and class split, so their values here are digitized from their plots, with a reading uncertainty of about 0.02. In addition, the publicly downloadable MIRAGE-2019 archive contains two devices, and its per-app distribution differs from the 40-app capture described in their text, so the two evaluations do not use identical data. The comparison is therefore best-effort rather than controlled, and it shows that retrieval-based adaptation reaches competitive accuracy at a small fraction of the adaptation cost.

5.5. Out-of-distribution detection

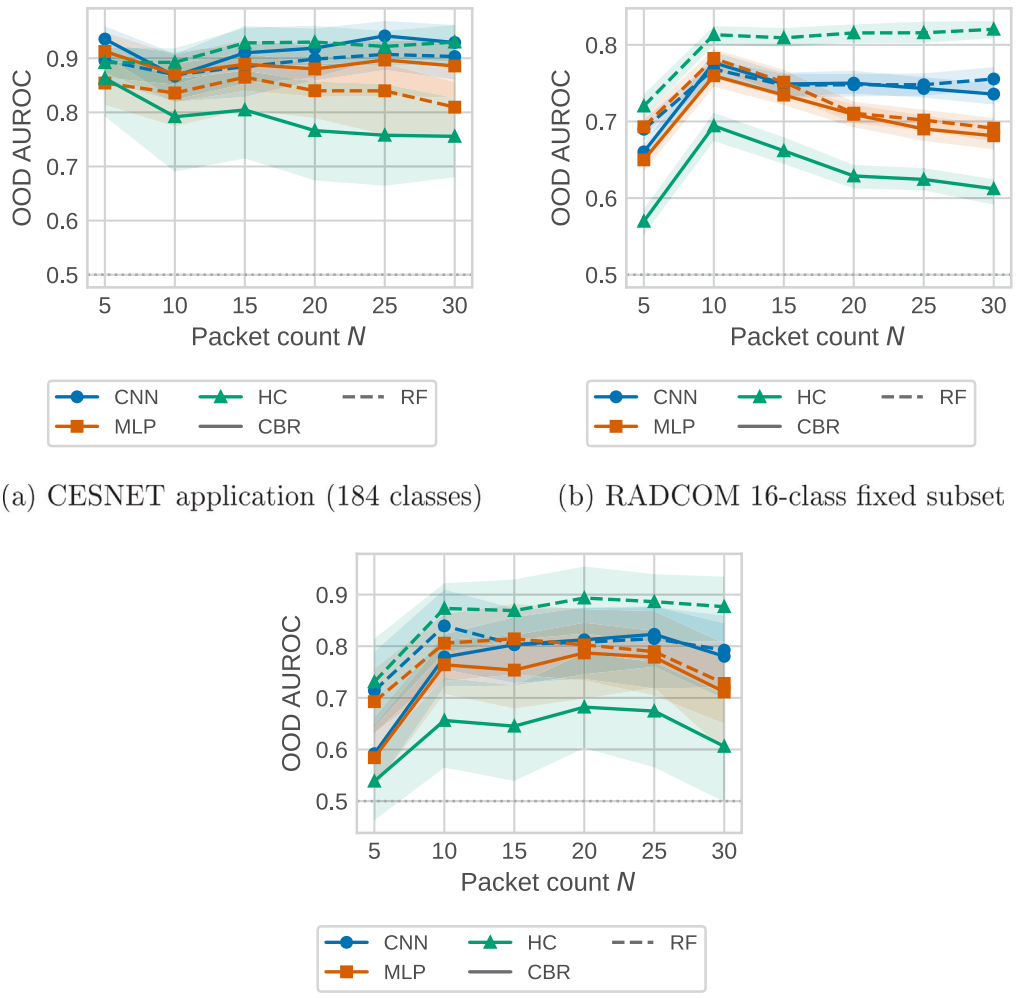
OOD detection is evaluated as binary AUROC. In-distribution samples are drawn from the base test set, while OOD samples are the unseen-class flows held out for few-shot adaptation, scored against an index that contains base classes only. Those scores do not depend on the shot budget K, because the index is the same in every case, so K governs only how many unseen-class flows are scored and not what they score. AUROC is a threshold-free metric that summarizes discriminative performance across all possible operating points without requiring the selection of a detection threshold. CBR uses L2 distance to the nearest indexed neighbor as the anomaly score (lower similarity indicates a likely OOD sample). RF uses one minus the maximum class probability as a proxy anomaly score.

The backbone ranking established in the classification experiments largely carries over to OOD detection. For CBR-based OOD detection on CESNET, the CNN backbone yields the highest average AUROC (0.917), followed by MLP (0.889) and HC (0.790) (Fig. 13). The strong CNN result indicates that the dense embedding space learned by the convolutional model produces well-separated clusters, allowing distance-based scores to identify OOD flows with high reliability. MLP ranks a close second, confirming that its learned embedding also separates in-distribution and out-of-distribution flows effectively across the full packet budget. HC features trail both learned backbones in CBR OOD detection, indicating that the HC feature space encodes enough structural variation to flag fundamentally different traffic patterns yet supports retrieval-based anomaly scoring less effectively than learned embeddings.

The RF-based OOD results expose a sharper contrast. For CNN and MLP backbones, CBR achieves comparable or better AUROC than RF on CESNET: CNN-CBR reaches 0.936 at $N = 5$ and maintains 0.867–0.941 across the full packet budget, exceeding CNN-RF (0.870–0.907) at every value. For HC features, however, RF substantially outperforms CBR, with HC-RF achieving 0.892–0.930 against HC-CBR at 0.756–0.863, a gap of nearly 13 points on average. This result mirrors the pattern observed in base accuracy and few-shot performance, confirming that HC features are more naturally exploited by tree-based methods in both in-distribution and out-of-distribution settings.

MIRAGE-2019 exhibits the same HC contrast (Fig. 13(c)): HC-RF reaches an average AUROC of 0.850 against 0.654 for HC-CBR, the widest backbone gap. For the deep backbones the two methods are level (0.790 vs. 0.785 for CNN, 0.776 vs. 0.761 for MLP), and the CBR ranking CNN > MLP > HC is preserved. Even the weakest configuration (HC-CBR) averages 0.654 AUROC, so every backbone separates the held-out MIRAGE apps from the base classes above the 0.5 chance level.

To characterize the sensitivity–specificity trade-off at a single operating point, Fig. 13 is complemented by the Youden-optimal threshold



(a) CESNET application (184 classes)

(b) RADCOM 16-class fixed subset

(c) MIRAGE-2019 (15 classes)

Fig. 13. Out-of-distribution detection AUROC as a function of the observed packet count N for CBR (solid) and RF (dashed) across the CNN, MLP, and HC backbones (color and marker per backbone). Higher is better, and the dotted line marks the 0.5 random-chance level. The CBR backbone ranking is CNN > MLP > HC, and HC-RF substantially outperforms HC-CBR on all three datasets. The shaded band is the 95% interval of each AUROC under repeated balanced resampling of the two score pools. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 7
Youden-optimal OOD operating points at $N = 15$. *Score thresh.* is the Elasticsearch similarity score below which a flow is flagged OOD, τ_{ℓ_2} is the equivalent ℓ_2 distance, *Sens.* is OOD recall, and *Spec.* is base-class retention rate.

Dataset	Model	Score thresh.	τ_{ℓ_2}	Sens.	Spec.
CESNET	CNN-CBR	0.962151	0.1983	93.6%	79.1%
	MLP-CBR	0.994885	0.0717	85.1%	81.9%
	HC-CBR	4.7×10^{-5}	146.1233	91.5%	63.2%
RADCOM	CNN-CBR	0.995732	0.0655	92.4%	47.6%
	MLP-CBR	0.999364	0.0252	89.0%	51.9%
	HC-CBR	5.2×10^{-5}	138.0433	87.4%	46.3%
MIRAGE	CNN-CBR	0.972499	0.1682	70.0%	88.3%
	MLP-CBR	0.999622	0.0195	72.5%	76.8%
	HC-CBR	9.8×10^{-4}	31.9624	80.0%	51.5%

for each backbone at $N = 15$ (Table 7), i.e., the score below which a flow is flagged OOD that maximizes sensitivity + specificity −1 (the ROC point farthest from the random-chance diagonal). *Score thresh.* is the Elasticsearch similarity score at that point, τ_{ℓ_2} is the equivalent ℓ_2 distance, *Sens.* is OOD recall (fraction of OOD flows correctly detected), and *Spec.* is base-class precision (fraction of in-distribution flows correctly retained).

CNN-CBR and MLP-CBR reach balanced operating points, each exceeding 85% sensitivity and 79% specificity. The very large threshold for HC-CBR ($\tau = 146$) reflects that HC features in $17 + N$ dimensions span a far larger ℓ_2 scale than learned embeddings, so a distance that counts as remote for an embedding is unremarkable for an HC vector, and the two score distributions overlap much more heavily. The full ROC curves at $N = 15$ are shown in Fig. 14(a): CNN-CBR and MLP-CBR trace well-separated curves reaching the upper-left quadrant, while HC-CBR stays closer to the diagonal throughout, consistent with the wider overlap implied by Table 7.

The RADCOM dataset fully corroborates these OOD findings (Fig. 13). The CBR backbone ranking remains CNN > MLP > HC, with average AUROC values of 0.736, 0.704, and 0.632 respectively. The HC RF advantage over HC CBR is even more pronounced on RADCOM: HC-RF reaches 0.720–0.820 against HC-CBR at 0.570–0.694, a gap of nearly 17 points on average. This widening of the HC deficit reinforces the conclusion that HC statistical features support RF-based OOD detection far more effectively than retrieval-based methods, and that this structural incompatibility is a robust property of the HC feature representation rather than a dataset-specific artifact.

Table 7 shows the RADCOM Youden-optimal operating points. HC-CBR shows the same very large distance scale as observed on CESNET

($\tau > 100$), with the score distributions of in- and out-of-distribution flows overlapping substantially.

CNN-CBR and MLP-CBR require very high score thresholds (score > 0.99, $\tau < 0.07$), meaning an OOD flow must be an almost-perfect nearest-neighbor match to avoid being flagged. While sensitivity exceeds 87% for all three models, specificity drops to approximately 46%–52%. This is lower than on CESNET because the smaller 16-class base set provides less discriminative margin between in- and out-of-distribution flows. Similarly, HC-CBR yields a very large distance threshold ($\tau = 138$) for the same reason: HC feature vectors in $17 + N$ dimensions span a far larger ℓ_2 scale than learned embeddings. The corresponding ROC curves for RADCOM (Fig. 14(b)) confirm these patterns, with CNN-CBR and MLP-CBR maintaining clear separation from the diagonal while HC-CBR runs closest to chance level.

MIRAGE-2019 is the only dataset where the Youden point shifts toward the opposite trade-off. Sensitivity exceeds specificity in all six CESNET and RADCOM configurations, by between 3 and 45 points, prioritizing OOD detection over base retention. On MIRAGE this ordering reverses for both deep backbones, by 18 points for CNN-CBR and 4 for MLP-CBR, while HC-CBR keeps the majority ordering. Two properties of the split account for the reversal. First, its compact 15-class base set brings in-distribution flows closer to their neighbors. Second, the held-out apps are mainstream consumer services rather than background traffic, pulling the score distributions closer together. Consequently, the optimal point adjusts to preserve base-class retention, most visibly for CNN-CBR, which retains 88.3% of base-class flows at 70.0% OOD recall. HC-CBR again yields a threshold orders of magnitude above the learned backbones ($\tau = 32.0$), though roughly a quarter of its scale on the other datasets.

5.5.1. Reading the ROC curves and the effect of the threshold

The Youden point above is one operating point, but a deployment is free to choose any other, so it is worth stating precisely what the ROC curves in Fig. 14 contain. Each curve is generated by sweeping the detection threshold over the whole range of the anomaly score, and every point on the curve is the (false positive rate, true positive rate) pair produced by one such threshold. The threshold therefore does not move the curve, it selects a position along it. The horizontal axis is the fraction of in-distribution flows wrongly flagged, which is one minus specificity, and the vertical axis is the fraction of unseen-class flows correctly flagged, which is sensitivity or OOD recall. The curves are the per-threshold result in full, and AUROC is simply the area beneath them. The shaded band around each curve is a 95% interval over both score pools. The curves are drawn at $N = 15$, and the equivalent sweep at every other packet count is reported in the supplementary material rather than here, so an operating point can be read off at whichever packet budget a deployment uses.

Because a raw score threshold is not comparable across representations that place their scores on different scales, Table 8 reports the same curves at five fixed *specificity* levels, that is, at thresholds that retain a prescribed fraction of base-class traffic. This is the quantity an operator actually controls: it fixes how much legitimate traffic may be diverted for inspection and asks what share of unseen traffic is caught in return. The trade-off is steep. On CESNET, CNN-CBR detects 89.4% of OOD flows while retaining 80% of base-class traffic, but tightening the threshold to retain 95% drops detection to 38.3%, so protecting a further 15% of in-distribution flows costs more than half of the OOD recall. The same pattern holds on every dataset and for both classifiers, and it is the reason a single reported sensitivity is meaningless without the specificity it was measured at.

The table also makes the HC deficit concrete. At 80% retention on CESNET, HC-CBR detects 57.4% of OOD flows against 89.4% for HC-RF, and the equivalent distance threshold is $\tau \approx 524$ against $\tau \approx 0.21$ for CNN-CBR. The HC space is not uninformative, since HC-CBR still reaches 0.805 AUROC, well above chance, but its in- and OOD score distributions overlap enough that no threshold separates them cleanly, which is what the near-diagonal HC-CBR curve in Fig. 14(a) shows. Comparing the two classifiers across the sweep sharpens the earlier AUROC reading: CBR leads RF at every operating point with the CNN backbone on CESNET, the two are near-indistinguishable on RADCOM, and RF leads with HC features on all three datasets and on MIRAGE with every backbone.

5.5.2. Cross-dataset malware detection

The OOD experiments so far withhold benign application classes from the index and ask whether their flows are recognized as unfamiliar. A deployment, however, must also flag traffic that is unfamiliar in a stronger sense, coming from a different population altogether. To test this we take the malware flows of the MTA dataset (Section 4.1) and present them, unchanged, to the CESNET-CBR and the CESNET-RF. Neither model has seen malware of any kind. This is the only cross-dataset evaluation in the paper: the in-distribution pool is the CESNET base test set and the OOD pool is roughly 29,800 malware flows across seven families, so the anomaly the detector must catch is a genuine domain shift rather than a held-out label.

With learned embeddings the malware is separated far more cleanly than the held-out applications were. Fig. 15 shows CNN-CBR averaging 0.996 AUROC across the packet budget and MLP-CBR 0.980, both above the 0.917 CNN-CBR figure for held-out CESNET applications. The retrieval geometry that made held-out apps only moderately separable makes malware, which shares almost none of the structure of the indexed benign classes, sit far from every base cluster. The Isolation Forest baseline is also strong for these backbones (CNN-RF averages 0.987 and MLP-RF 0.966), which is expected when the two populations differ this much, and it confirms that the separation is a property of the traffic and not an artifact of the retrieval scoring.[2]

HC features do not transfer nearly as well: HC-CBR averages 0.866 and HC-RF 0.875, some 11 points below the learned backbones. This repeats, in a harder setting, the coupling reported throughout this paper between feature geometry and the downstream classifier. The HC space spreads benign traffic so widely that malware is no longer remote from it in relative terms, and the tree ensemble again exploits that space better than retrieval does. HC-CBR is also the only curve in Fig. 15 that depends materially on the packet budget, falling from 0.961 at $N = 5$ to 0.826 at $N = 30$ while every learned curve stays flat. The extra packet sizes appear to add dimensions in which benign traffic and malware overlap, so each one dilutes the HC distance rather than sharpening it. A short observation window is therefore not a handicap for this probe, which matters operationally because it is the regime a real detector runs in.

The operating points make the practical margin concrete. Table 9 reports malware detection against the fraction of benign traffic retained, at $N = 15$, the packet count Table 8 also uses. CNN-CBR detects 99.4% of malware flows while retaining 95% of benign traffic, and still 81.8% of them at the stringent 99% retention level, where detection of merely held-out applications falls to 12.8% at the same packet count. The HC operating points hold up while the threshold is permissive, 99.9% detection at 70% retention for HC-CBR, then collapse once it tightens, to 34.9% at 90% retention, 4.4% at 95% and nothing at all at 99%, so the headline result belongs to the learned backbones alone. As stated in Section 3.3, Isolation Forest is included here only to check that the retrieval step detects malware relative to a standard anomaly detector, not to argue superiority over dedicated OOD methods. The result to take away is that with a learned embedding the same distance threshold already computed for classification separates unknown malware from benign application traffic almost perfectly, at no additional modeling cost.

[2] A retrieval score is a distance to a particular index and is only comparable with in-distribution scores drawn from the same index, so the two families are reported on their own bases rather than pooled.

Table 8
OOD detection across the threshold range at $N = 15$. Each column fixes the detection threshold at the score that retains a prescribed fraction of in-distribution (base-class) traffic, and reports the resulting sensitivity, the percentage of unseen-class flows detected. Reading a row left to right traces one curve of Fig. 14 from a permissive threshold to a strict one. AUROC is the threshold-free area under that curve. Detection falls steeply as the threshold tightens, so a sensitivity figure is only meaningful together with the specificity at which it was obtained. RF probabilities are quantized to 1/100 by the 100-tree ensemble, so an RF threshold cannot always be placed exactly at its target: the retention achieved differs from the column heading by up to 0.8 points. CBR scores are continuous and meet every target to within 0.1 points.

Dataset	Model	AUROC	Sensitivity (%) at base-class retention				
			70%	80%	90%	95%	99%
CESNET	CNN-CBR	0.910	97.9	89.4	66.0	38.3	12.8
	CNN-RF	0.884	89.4	85.1	59.6	38.3	10.6
	MLP-CBR	0.889	93.6	85.1	61.7	38.3	10.6
	MLP-RF	0.865	87.2	87.2	63.8	40.4	8.5
	HC-CBR	0.805	68.1	57.4	29.8	14.9	2.1
	HC-RF	0.928	91.5	89.4	83.0	68.1	17.0
RADCOM	CNN-CBR	0.749	66.7	46.2	24.5	12.9	3.0
	CNN-RF	0.747	68.4	47.4	23.0	10.6	1.7
	MLP-CBR	0.734	61.1	41.0	22.8	11.0	2.4
	MLP-RF	0.751	66.8	45.7	20.0	9.2	2.4
	HC-CBR	0.662	42.2	27.3	9.5	3.5	0.8
	HC-RF	0.809	78.0	63.0	32.9	18.4	2.1
MIRAGE	CNN-CBR	0.803	80.0	75.0	50.0	30.0	5.0
	CNN-RF	0.804	80.0	57.5	37.5	20.0	5.0
	MLP-CBR	0.754	75.0	67.5	40.0	17.5	2.5
	MLP-RF	0.814	80.0	67.5	50.0	22.5	5.0
	HC-CBR	0.645	52.5	35.0	15.0	7.5	0.0
	HC-RF	0.869	90.0	80.0	47.5	32.5	10.0

Table 9
Cross-dataset malware detection across the threshold range at $N = 15$, the packet count of Table 8. The CESNET index and Random Forest, trained only on benign application traffic, score MTA malware flows they have never seen. Each column fixes the detection threshold at the score that retains a prescribed fraction of benign base-class traffic, and reports the resulting sensitivity, the percentage of malware flows detected. Isolation Forest is included as a reference baseline to show that the retrieval step detects malware, not as a comparison against dedicated OOD methods. RF probabilities are quantized to 1/100 by the 100-tree ensemble, so an RF threshold cannot always be placed exactly at its target: the retention achieved differs from the column heading by up to 0.6 points.

Model	AUROC	Malware detected (%) at benign retention				
		70%	80%	90%	95%	99%
CNN-CBR	0.993	100.0	100.0	99.9	99.4	81.8
CNN-RF	0.986	100.0	100.0	99.6	95.2	55.5
MLP-CBR	0.985	100.0	99.7	99.2	94.9	63.9
MLP-RF	0.967	99.6	98.8	89.0	81.1	35.2
HC-CBR	0.856	99.9	73.6	34.9	4.4	0.0
HC-RF	0.891	99.5	92.5	55.4	14.9	0.3

6. Discussion and limitations

In this section, we revisit our core contributions (Section 1.1) and contextualize them within the inherent trade-offs and limitations of our approach. Rather than viewing successes and limitations in isolation, we discuss how the design choices that enable our framework's capabilities simultaneously define its boundaries.

Adaptability vs. Serving Latency. The primary advantage of our framework is its ability to adapt to new classes without retraining (Contribution 1) and the sheer efficiency of its index construction (Contribution 3). As shown in Section 5.4, inserting a handful of labeled vectors into the running index absorbs new classes in seconds, requiring only the first few packets of a flow (Section 5.3). On CESNET, index building is roughly ten times faster than full RF retraining (Fig. 7), and MIRAGE-2019 reproduces this: absorbing an unseen application costs 34 to 37 s for the deep backbones and 7 to 10 s for HC, against 187 to 1310 s to retrain the forest. On that benchmark, we achieve comparable performance to the strongest of the eight methods of Bovenzi et al. [22] at $K = 5$ (and within 3.5 points at all other budgets), while outperforming the remaining seven baselines at $K = 50$ and $K = 100$ (Fig. 12). Only one of those eight, MatchingNet, absorbs a new application without an optimization step, so we reach that accuracy while removing every training stage rather than in spite of one. However, this extreme adaptability comes with an inference-time trade-off. As detailed in Section 5.2, per-flow serving is slower than a depth-limited tree ensemble. On CESNET an in-process RF classifies an individual flow 4.6 to 16 times faster than a retrieval query, depending on the feature backbone, and the margin is wider still on the smaller datasets, reaching 0.017 ms against 0.198 ms for the retrieval step alone on MIRAGE (Table 5). Furthermore, CBR relies on ANN search (HNSW), which trades a small amount of accuracy for query speed. A deployment whose absolute binding constraint is per-flow latency rather than adaptability would be better served by a traditional tree ensemble.

Unified OOD Detection and Class-Space Dependency. By embedding the classification process in a metric space, the same retrieval step provides a unified, training-free mechanism for OOD detection (Contribution 2). CNN-CBR averages 0.917 AUROC on CESNET using a single global distance threshold (Fig. 13). Yet, the effectiveness of

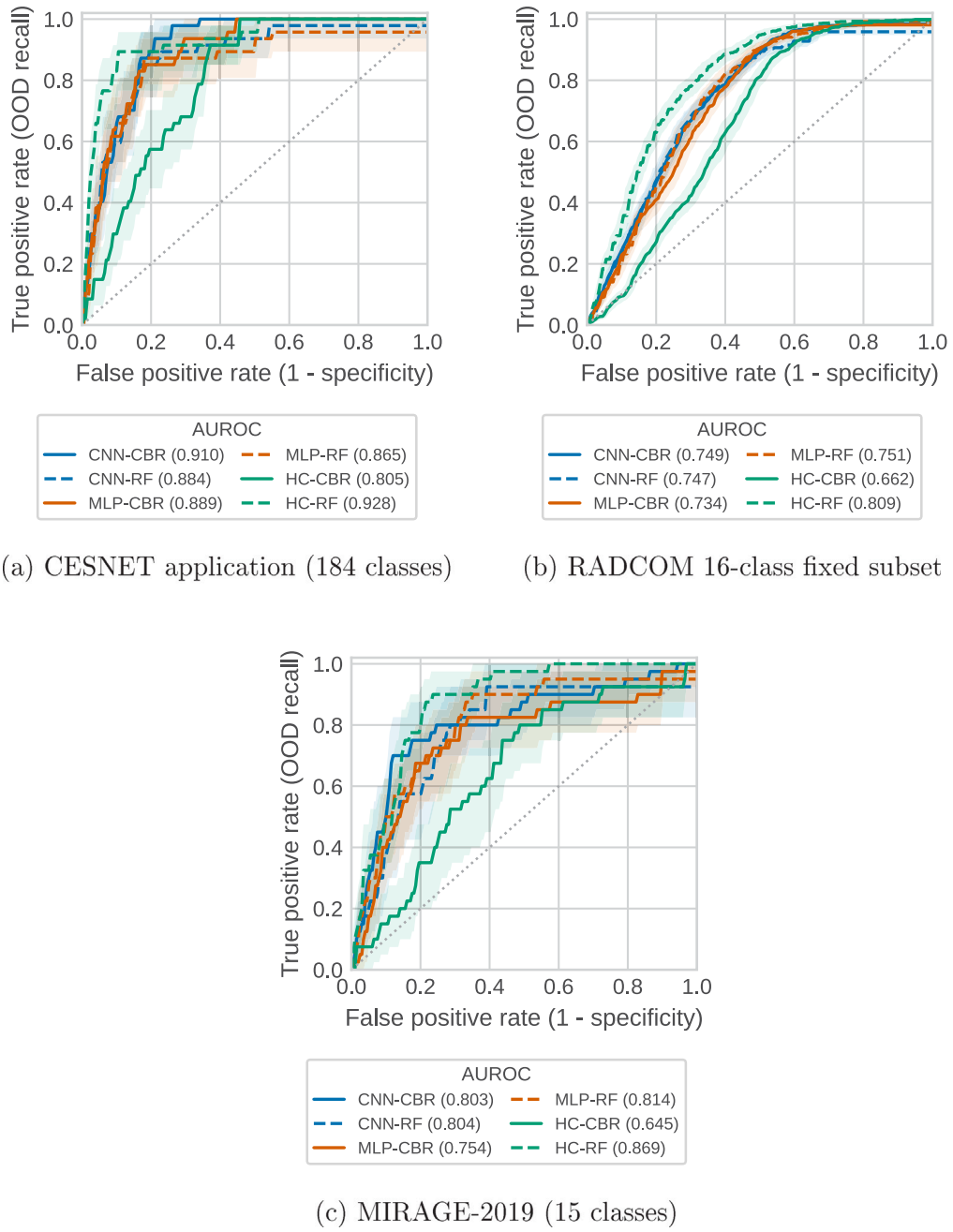


Fig. 14. ROC curves for out-of-distribution detection at $N = 15$, for CBR (solid) and RF (dashed) across the CNN, MLP, and HC backbones (color per backbone). Every point on a curve corresponds to one detection threshold: the threshold selects an operating point along the curve rather than changing the curve itself, so these three panels contain the complete per-threshold result. The horizontal axis is the false positive rate, the fraction of in-distribution flows wrongly quarantined, and the vertical axis is the true positive rate, the fraction of out-of-distribution flows detected. The dotted diagonal marks random chance and AUROC is the area under each curve. Numeric operating points read off these curves are given in Table 8. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

this spatial separation is highly sensitive to the size and diversity of the known class space. On the smaller 16-class RADCOM subset, OOD specificity drops to roughly 46%–52%. A smaller base set offers a narrower discriminative margin between in- and out-of-distribution flows, meaning our framework's relative benefit is most pronounced on large, highly diverse class spaces. The 15-class MIRAGE base set confirms this, with CBR averages of 0.785 for CNN, 0.761 for MLP and 0.654 for HC, and is the only dataset whose Youden point reverses to favor specificity (Section 5.5). Base-set size is not the whole story, however. What the threshold really measures is how far the unknown population sits from the indexed one: presented to that same 184-class CESNET index, MTA malware yields 0.996 AUROC for CNN-CBR against 0.917 for held-out applications, detecting 99.4% of malware flows at 95% benign retention (Section 5.5.2). The mechanism is weakest where an unknown class neighbors a known one, and strongest on genuine domain shift, the case an operational detector is deployed to catch.

Feature Representation and Geometric Strictness. By evaluating across the feature spectrum (Contribution 4), we demonstrated how HC statistical features and learned embeddings (CNN, MLP) interact with different classification paradigms across packet budgets and few-shot regimes. This comparison revealed a strict dependency limitation: no

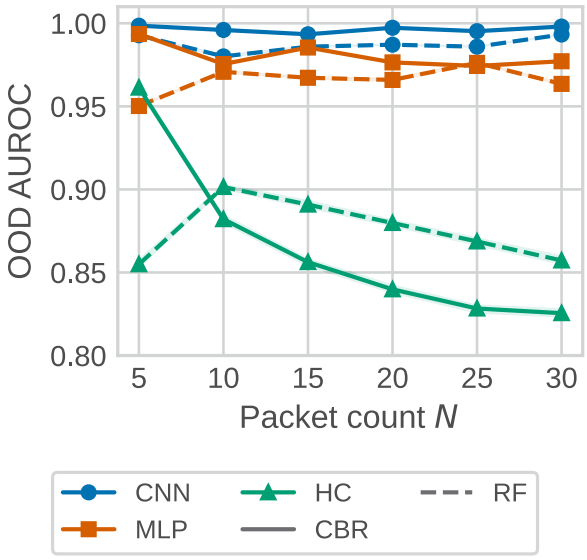


Fig. 15. Cross-dataset malware detection: out-of-distribution AUROC as a function of the observed packet count N when the CESNET index and Random Forest, trained only on benign application traffic, score MTA malware flows they have never seen. CBR is solid and RF (Isolation Forest) dashed, with color and marker per backbone. The shaded bands are 95% intervals, narrow here because both pools are large. The learned backbones sit near the ceiling and stay flat across the packet budget, well above the held-out-application results of Fig. 13, while the HC runs trail by roughly 11 points, so the vertical axis starts at 0.80. HC-CBR is the only series that varies materially with N, falling from 0.961 at $N = 5$ to 0.826 at $N = 30$. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

single extraction method is universally optimal. Learned deep embeddings cluster effectively in metric space, making them naturally suited for our distance-based retrieval and OOD detection. Conversely, HC statistical features remain highly competitive when paired with RF, but perform poorly under retrieval due to overlapping boundaries in the raw feature space. This rigid coupling between feature geometry and downstream classification remains a broader challenge in encrypted traffic analysis. The coupling holds on every dataset we examined. MIRAGE produces the widest backbone gap of the study for OOD detection, 0.850 for HC-RF against 0.654 for HC-CBR, alongside a comparable split in base classification, 0.824 against 0.651. MTA shows that it survives a full domain shift as well: HC transfers some 11 points below the learned backbones, at 0.866 for HC-CBR and 0.875 for HC-RF against 0.996 and 0.980. That probe also exposes a failure mode the in-distribution experiments do not. HC-CBR is the only configuration whose AUROC falls as the observation window grows, from 0.961 at $N = 5$ to 0.826 at $N = 30$, while every learned curve stays flat. Each additional packet size contributes a dimension on which benign and malicious traffic overlap, diluting the distance rather than sharpening it. With HC features, observing more of a flow is therefore not uniformly beneficial, and the packet budget has to be treated as a parameter to be tuned rather than maximized.

System Scalability and Evaluation Scope. Finally, the memory footprint of the framework scales differently than traditional models. The HNSW index must remain resident in memory, growing linearly in the number of indexed flows at a per-flow cost set by the vector width, from roughly 120 bytes at $N = 5$ to 230 bytes at $N = 30$. At any given packet count that cost is nearly identical across the three classification datasets and is independent of the class count, which is precisely the axis along which an operational classifier is expected to expand. While our measurements show that the index is actually much leaner than a serialized RF (which grows exponentially with class count), a massive corpus far exceeding those studied here would eventually require sharding. Regarding the scope of our study, we evaluate two public datasets (CESNET-TLS22 with 184 classes and MIRAGE-2019 with 23 mobile-application classes), one proprietary cellular dataset (RADCOM), and a fourth corpus of roughly 29,800 malware flows (MTA) used solely as an OOD probe. Cross-dataset transfer is therefore established for OOD detection, where an index built on CESNET flags malware drawn from an unrelated capture it has never seen, but not for classification: we do not test whether an index built on one network classifies flows captured on another, and all three classification datasets carry TLS or QUIC application traffic. Cross-network classification transfer, non-TLS traffic domains, and drift over horizons longer than Section 5.3 covers all remain open.

7. Conclusion and future work

We presented an ultra-fast encrypted traffic classification system based on Classification By Retrieval. Each flow is represented by a fixed-length vector extracted from only the first few packets, indexed in an HNSW ANN structure, and classified by retrieving its nearest labeled neighbor, while out-of-distribution traffic is detected within the same step using a single global distance threshold. With CNN embeddings, CBR matches the RF baseline in accuracy (macro F1 of 0.923 versus 0.925 on CESNET-TLS22) while training roughly ten times faster, adapts to new classes two to three orders of magnitude faster, and detects out-of-distribution flows at up to 0.92 AUROC.

The practical advantage of retrieval-based classification grows with scale. When the number of classes is large and the total volume of labeled data is high, even if individual classes are not, parametric classifiers must pay a retraining cost that grows with the entire dataset, whereas CBR absorbs new data and new classes through indexing alone. In operational settings that demand fast and automatic adaptation, this asymmetry between cheap indexing and expensive retraining is a decisive and clearly felt benefit.

Several directions remain open. First, closing the HC-feature gap through better metric learning or learned re-weighting, and studying the candidate-pool size as an explicit accuracy-latency trade-off, would broaden the framework's applicability. Second, hybrid representations that combine HC statistics with learned embeddings may capture complementary signal. Third, having established cross-dataset transfer for OOD detection, the corresponding question for classification is open: whether an index built on one network classifies flows captured on another. That, together with adaptive or per-class OOD thresholds and evaluation on streaming deployments under concept drift,forms the natural next step toward production use.

A further direction targets per-flow inference cost, the primary dimension where retrieval underperforms the baseline. Section 5.2 shows that retrieval time increases with embedding width, ranging from 4.6× slower than RF with HC features to 14×–16× slower with learned backbones, whereas a depth-limited forest is largely insensitive to input width. This points to two unexamined optimizations. First, the embedding dimension can be reduced; while we report results for $d = 128$, evaluating smaller dimensions $d \in \{16, 32, 64\}$ could proportionally decrease query cost while preserving accuracy and leaving the RF baseline unaffected. Second, client-side overhead accounts for half of the end-to-end latency, specifically vector construction and the network round-trip to Elasticsearch, and eliminating it via an embedded index would mitigate this delay. Since neither optimization impacts memory footprint or adaptation performance, both represent viable enhancements to address this limitation.

CRediT authorship contribution statement

Yehonatan Zion: Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Amit Dvir:** Writing – review & editing, Supervision, Conceptualization. **Chen Hajaj:** Writing – review & editing, Supervision, Methodology, Formal analysis. **Ran Dubin:** Writing – review & editing, Supervision, Conceptualization. **Neriya Filber:** Validation, Software, Investigation.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Ran Dubin has patent # 63/448,460 pending to Ariel Scientific Innovations LTD (ASI). If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

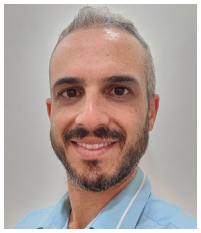
This work was supported by the Ariel Cyber Innovation Center, the Israel Innovation Authority, and the ENTM consortium. The work is patent-based on US Provisional Patent Application No. 63/448,460.

Data availability

Multiple data sources were used: one is unavailable due to privacy reasons, the others are available.

References

[1] Internet Security Research Group, Let's encrypt, 2021, URL https://letsencrypt.org/about/.
[2] Google, QUIC, a multiplexed transport over UDP, 2023, URL https://www.chromium.org/quic/.
[3] T. Böttger, F. Cuadrado, G. Antichi, E.L. Fernandes, G. Tyson, I. Castro, S. Uhlig, An empirical study of the cost of DNS-over-HTTPS, in: Proceedings of the Internet Measurement Conference, 2019, pp. 15–21.
[4] Ericsson, Ericsson mobility report, 2024, URL https://www.ericsson.com/en/reports-and-papers/mobility-report/dataforecasts/mobile-traffic-forecast. (Accessed 22 August 2024).
[5] R. Enisoglu, B.A. Inan, Network traffic classification from handcrafted features to foundation models: A comprehensive survey, Comput. Netw. 287 (2026) 112578, http://dx.doi.org/10.1016/j.comnet.2026.112578.
[6] S. Han, H. Zhang, M. Lu, S. Guo, B. Tian, J. Wang, A comprehensive survey on encrypted network traffic classification, Comput. Netw. 287 (2026) 112524, http://dx.doi.org/10.1016/j.comnet.2026.112524, URL https://www.sciencedirect.com/science/article/pii/S1389128626005360.
[7] Y.A. Malkov, D.A. Yashunin, Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, IEEE Trans. Pattern Anal. Mach. Intell. 42 (4) (2020) 824–836, http://dx.doi.org/10.1109/TPAMI.2018.2889473.
[8] W. Wang, M. Zhu, X. Zeng, X. Ye, Y. Sheng, Malware traffic classification using convolutional neural network for representation learning, in: 2017 International Conference on Information Networking, ICOIN, IEEE, 2017, pp. 712–717.
[9] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: 2017 IEEE International Conference on Intelligence and Security Informatics, ISI, IEEE, 2017, pp. 43–48.
[10] C. Liu, L. He, G. Xiong, Z. Cao, Z. Li, Fs-net: A flow sequence network for encrypted traffic classification, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, IEEE, 2019, pp. 1171–1179.
[11] P. Wang, X. Chen, F. Ye, Z. Sun, A survey of techniques for mobile service encrypted traffic classification using deep learning, IEEE Access 7 (2019) 54024–54033.
[12] V.F. Taylor, R. Spolaor, M. Conti, I. Martinovic, Robust smartphone app identification via encrypted network traffic analysis, IEEE Trans. Inf. Forensics Secur. 13 (1) (2017) 63–78.
[13] A.J. Pinheiro, J.d.M. Bezerra, C.A. Burgardt, D.R. Campelo, Identifying IoT devices and events based on packet length from encrypted traffic, Comput. Commun. 144 (2019) 8–17.
[14] M. Zhan, R. Zhao, X. Deng, Z. Xue, Q. Li, Z. Liu, G. Cheng, K. Xu, FlowRefiner: A robust traffic classification framework against label noise, Adv. Neural Inf. Process. Syst. 38 (2026) 128242–128265.
[15] A. Biernacki, Evaluating filter, wrapper, and embedded feature selection approaches for encrypted video traffic classification, Electronics 14 (18) (2025) 3587.
[16] Y. Zhao, G. Dettori, M. Boffa, L. Vassio, M. Mellia, The sweet danger of sugar: Debunking representation learning for encrypted traffic classification, in: Proceedings of the ACM SIGCOMM 2025 Conference, 2025, pp. 296–310.
[17] C. Ma, X. Du, L. Cao, Improved KNN algorithm for fine-grained classification of encrypted network flow, Electronics 9 (2) (2020) 324.
[18] P. Ea, Q. Vo, O. Salem, A. Mehaoua, Is deep learning a better option than random forest for encrypted traffic classification? in: 2024 IEEE 49th Conference on Local Computer Networks, LCN, IEEE, 2024, pp. 1–6.
[19] A. Safari Khatouni, N. Seddigh, B. Nandy, N. Zincir-Heywood, Machine learning based classification accuracy of encrypted service channels: Analysis of various factors, J. Netw. Syst. Manage. 29 (1) (2021) 8.
[20] R.T. Elmaghraby, N.M.A. Aziem, M.A. Sobh, A.M. Bahaa-Eldin, Encrypted network traffic classification based on machine learning, Ain Shams Eng. J. 15 (2) (2024) 102361.
[21] D. Shamsimukhametov, A. Kurapov, M. Liubogoshchev, E. Khorov, Early traffic classification with encrypted clienthello: A multi-country study, IEEE Access 12 (2024) 142979–142993.
[22] G. Bovenzi, D. Di Monda, A. Montieri, V. Persico, A. Pescapé, Few shot learning approaches for classifying rare mobile-app encrypted traffic samples, in: IEEE INFOCOM 2023-IEEE Conference on Computer Communications Workshops, INFOCOM WKSHPS, IEEE, 2023, pp. 1–6.
[23] G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, A. Pescapè, MIRAGE: Mobile-app traffic capture and ground-truth creation, in: IEEE 4th International Conference on Computing, Communication and Security, ICCCS 2019, 2019.
[24] J. Snell, K. Swersky, R. Zemel, Prototypical networks for few-shot learning, Adv. Neural Inf. Process. Syst. 30 (2017).
[25] O. Vinyals, C. Blundell, T. Lillicrap, D. Wierstra, et al., Matching networks for one shot learning, Adv. Neural Inf. Process. Syst. 29 (2016).
[26] Z. Li, J. Wang, Y.-F. Song, S.-H. Yue, Unlocking few-shot encrypted traffic classification: A contrastive-driven meta-learning approach, Electronics 14 (21) (2025) 4245.
[27] L. Zhao, C. Wu, A. Yu, When pre-training meets contrast learning: Few-shot encrypted traffic classification with novelty detection, IEEE Trans. Netw. (2026).
[28] C. Yang, G. Xiong, Q. Zhang, J. Shi, G. Gou, Z. Li, C. Liu, Few-shot encrypted traffic classification via multi-task representation enhanced meta-learning, Comput. Netw. 228 (2023) 109731.
[29] D. Monda, G. Bovenzi, A. Montieri, V. Persico, A. Pescapa, Analyzing the impact of shifts in encrypted mobile-app traffic on multimodal few-shot learning, Comput. Netw. (2025) 111935.
[30] W. Cai, C. Hou, M. Cui, B. Wang, G. Xiong, G. Gou, Incremental encrypted traffic classification via contrastive prototype networks, Comput. Netw. 250 (2024) 110591.
[31] W. Zhu, X. Ma, Y. Jin, R. Wang, ILETC: Incremental learning for encrypted traffic classification using generative replay and exemplar, Comput. Netw. 224 (2023) 109602.
[32] A. Ansari, K. Wang, P. Xiong, Out-of-distribution aware classification for tabular data, in: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, 2024, pp. 65–75.
[33] I. Meiri, R. Dubin, A. Dvir, C. Hajaj, Out-of-distribution is not magic: The clash between rejection rate and model success, in: 2025 20th Conference on Computer Science and Intelligence Systems, FedCSIS, IEEE, 2025, pp. 345–350.
[34] F.T. Liu, K.M. Ting, Z.-H. Zhou, Isolation forest, in: 2008 Eighth IEEE International Conference on Data Mining, IEEE, 2008, pp. 413–422.
[35] A. Hermans, L. Beyer, B. Leibe, In defense of the triplet loss for person re-identification, 2017, arXiv preprint arXiv:1703.07737.
[36] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, D. Krishnan, Supervised contrastive learning, Adv. Neural Inf. Process. Syst. 33 (2020) 18661–18673.
[37] elastic, What is elastic search? 2023, URL https://www.elastic.co/what-is/elasticsearch.
[38] J. Luxemburk, T. Čejka, Fine-grained TLS services classification with reject option, Comput. Netw. 220 (2023) 109467, http://dx.doi.org/10.1016/j.comnet.2022.109467, URL https://www.sciencedirect.com/science/article/pii/S1389128622005011.
[39] Radcom Ltd., Cellular network traffic dataset, 2025, Proprietary dataset. Radcom Ltd., Israel, Not publicly available; used under data-sharing agreement with traffic anonymized at collection.
[40] B. Duncan, Malware-traffic-analysis.net: A source for packet capture (pcap) files and malware samples, 2026, https://www.malware-traffic-analysis.net. (Accessed: 2026-07).
[41] Encrypted traffic classification: Supplementary results, 2026, https://anonymous.4open.science/r/etc-supplementary-results-8426/. Anonymized repository for review.
[42] K. Lee, S. Maji, A. Ravichandran, S. Soatto, Meta-learning with differentiable convex optimization, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 10657–10665.
[43] C. Finn, P. Abbeel, S. Levine, Model-agnostic meta-learning for fast adaptation of deep networks, in: International Conference on Machine Learning, PMLR, 2017, pp. 1126–1135.
[44] A. Raghu, M. Raghu, S. Bengio, O. Vinyals, Rapid learning or feature reuse? towards understanding the effectiveness of maml, 2019, arXiv preprint arXiv:1909.09157.
[45] F. Sung, Y. Yang, L. Zhang, T. Xiang, P.H. Torr, T.M. Hospedales, Learning to compare: Relation network for few-shot learning, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 1199–1208.



Yehonatan Zion is a Ph.D. student at Ariel University, supervised by Professor Amit Dvir, Professor Chen Hajaj, and Dr. Ran Dubin. Previously, he spent six years at Radcom, where he served as a Data Research Team Lead. During his tenure in the industry, he led the research and implementation of numerous machine learning systems and focused on data augmentation techniques to improve model performance. His expertise includes training AI models to predict Quality of Experience (QoE) on encrypted network traffic and enhancing the trustworthiness of AI models in telecommunications. Currently, he is focusing on his doctoral research, bridging large-scale industrial experience with academic advancements in machine learning, AI, and software engineering.



Chen Hajaj holds Ph.D. (Computer Science), M.Sc. (Electrical Engineering), and B.Sc. (Computer Engineering) degrees, all from Bar-Ilan University. He is a faculty member in the Department of Industrial Engineering and Management and a member of the Ariel Cyber Innovation Center. From 2016 to 2018, Chen was a postdoctoral fellow at Vanderbilt University. Chen's research activities are in the areas of Machine Learning, Game Theory, and Cybersecurity. Specifically, the focuses of his work are on encrypted traffic classification, how to detect and robustify the weak spots of AI methods (adversarial artificial intelligence), and Personalized and Preventative Medicine using data-science-based methods.



Amit Dvir received his B.Sc., M.Sc., and Ph.D. degrees from Ben-Gurion University, Beer Sheva, Israel, all in communication systems engineering. He is currently the Head of the Computer and Software Department and the Ariel Cyber Innovation Center at Ariel University, Israel. From 2011 to 2012, he was a Postdoctoral Fellow at the Laboratory of Cryptography and System Security, Budapest, Hungary. His research interests include the analysis of enrichment data from encrypted traffic. Prof. Dvir has published more than 50 research academic papers in top-tier conferences and journals.



Ran Dubin received his B.Sc., M.Sc., and Ph.D. degrees from Ben-Gurion University, Beer Sheva, Israel, all in communication systems engineering. He is currently a faculty member at the Computer Science Department, Ariel University, Israel. His research interests revolve around zero trust cyber protection, malware disarms and reconstruction, encrypted network traffic detection, Deep Packet Inspection (DPI), bypassing AI, Natural Language Processing (NLP), and AI trust enhancements.



Neriya Filber is an honors B.Sc. student in Computer Science, specializing in Data Science and Artificial Intelligence. Conducted under the supervision of Prof. Amit Dvir, Dr. Ran Dubin, and Mr. Yehonatan Zion, his research focuses on deep learning, Out-of-Distribution (OOD) detection, encrypted network traffic analysis, and trustworthy AI systems. In addition to his academic pursuits, Neriya brings practical industry experience from his current role at Wix, where he has worked for the past year. His hands-on approach bridges theoretical software development with real-world applications in cybersecurity and network communication.